$ W10 · RAN · FREE-RAN-UE · NR-DC

/> FREE-RAN-UE

從 CU / DU / RU、Small Cell、O-RAN，一路走進 free-ran-ue 原始碼：gNB、UE、四大流程，最後深入 NR-DC

/> 課程大綱

01
RAN
CU / DU / RU、Split、前傳

02
RAN TYPE
Small Cell、O-RAN、OCUDU

03
INTRO
free-ran-ue 定位、repo、設定

04
GNB
Start 順序、CP / DP 分工

05
UE
Start 順序、TUN、channel

06
PROCEDURE
NG Setup → 註冊 → PDU → 去註冊

07
NR-DC
Xn、Static / Dynamic，逐行讀 code

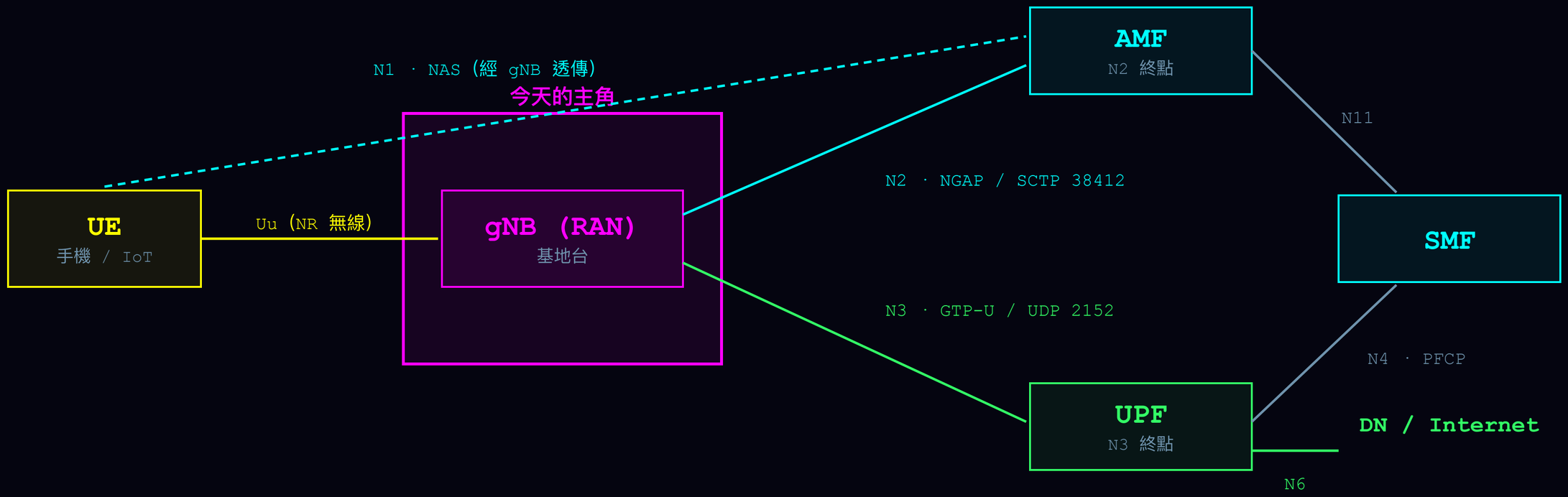
SOURCE

github.com/free-ran-ue/free-ran-ue

Part 1 真實 RAN（01-02） · Part 2 free-ran-ue 架構（03-05） · Part 3 流程與 NR-DC（06-07）

/> 回顧：RAN 在 5GS 的位置

一句話 gNB 夾在 UE 和核心網中間：往 AMF 走 N2、往 UPF 走 N3。



前幾週我們一直站在核心網這一側；今天把鏡頭轉到 gNB —— N2、N3 的另一端。

$ PART 1 · RADIO ACCESS NETWORK

/> RAN

Radio Access Network：把「無線電波」變成「核心網看得懂的訊令與封包」

/> RAN 在做什麼？

一句話 RAN 負責「無線那一段」，再把結果接上核心網。

01
無線接取
調變、編碼、波束成形，把位元變成電磁波（PHY / RF）

02
無線資源管理
每個 slot 決定誰用哪些 PRB、用多快（MAC Scheduler）

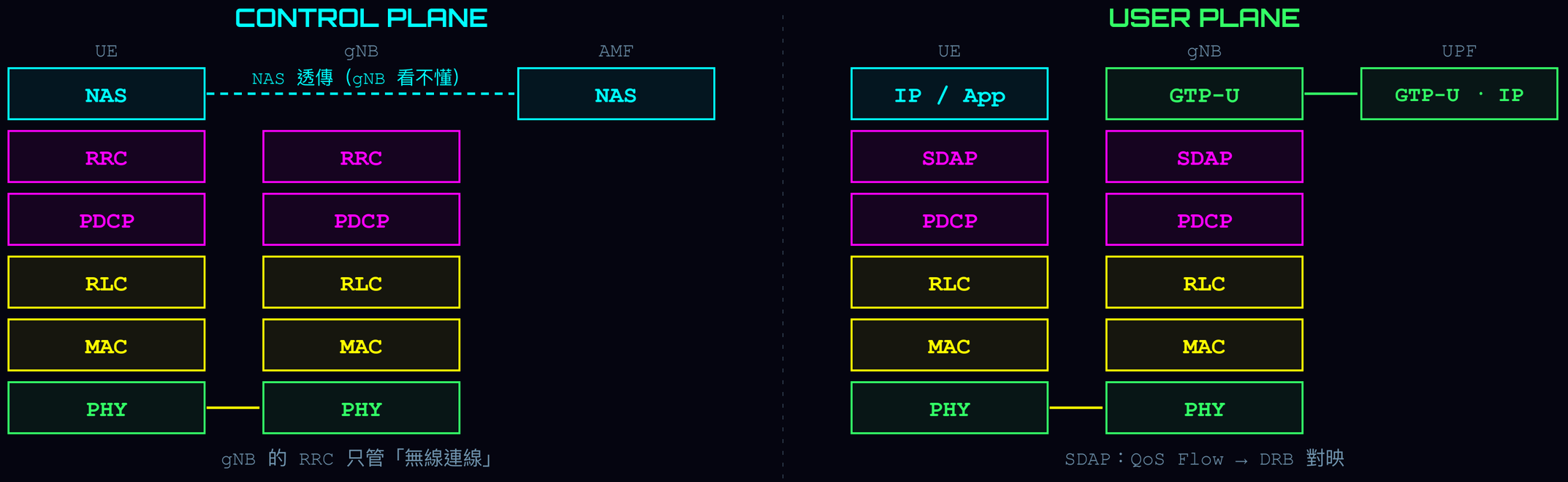
03
連線與移動性
RRC 連線建立、量測、Handover、Dual Connectivity

04
接上核心網
控制面走 N2 到 AMF，使用者面走 N3 到 UPF

前三項在空中介面那一側；第四項才是核心網（以及 free-ran-ue）真正在乎的部分。

NR 協定堆疊（Uu）

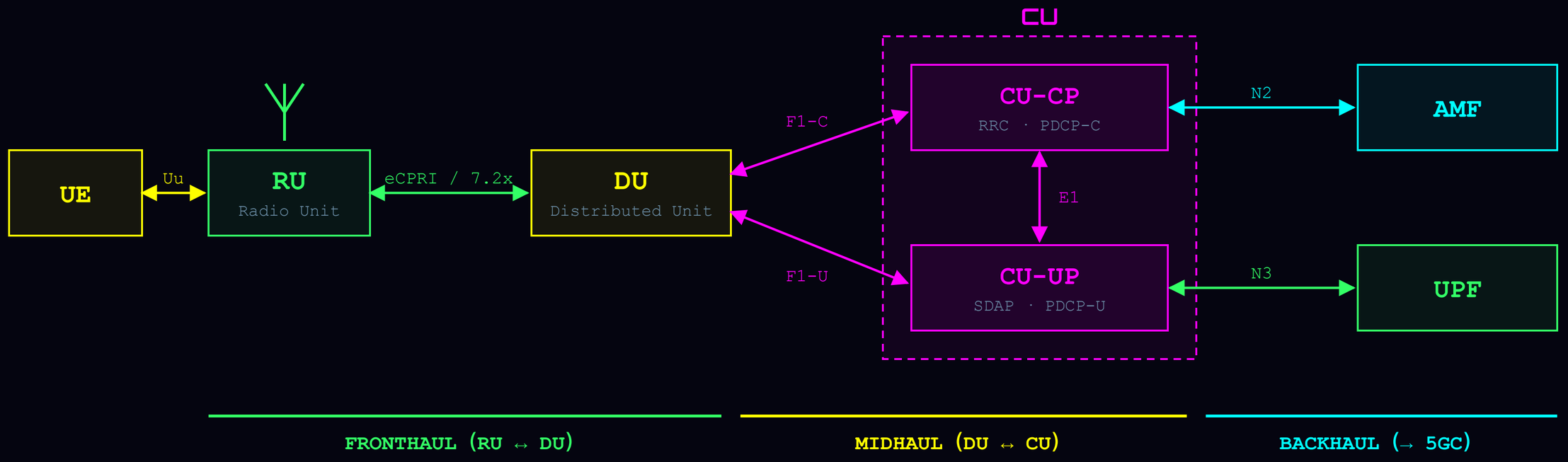
一句話 NAS 穿過 gNB 直達 AMF；其他各層都在 UE 和 gNB 之間。



放在 CU　放在 DU　PHY 跨 DU / RU　端到端（不在 RAN 處理）

GNB 拆成三塊：CU + DU + RU

一句話 一台 gNB 可以拆成 RU（天線）、DU（即時）、CU（非即時）三塊。



3GPP 定義 CU / DU（F1、E1）；DU / RU 之間的開放前傳則由 O-RAN 定義（7.2x）。

/> CU · CENTRALIZED UNIT

一句話 CU 管「不急的事」：連線、加密、QoS，可以集中放機房。

CU-CP

RRC · PDCP-C

RRC：連線建立、量測設定、Handover 決策

N2 對 AMF（NGAP）、F1-C 對 DU

CU-UP

SDAP · PDCP-U

SDAP：QoS Flow → DRB；PDCP：加密、重排序

N3 對 UPF（GTP-U）、F1-U 對 DU

非即時 → 可集中、可虛擬化

放在 edge DC，一個 CU 管多個 DU；CP / UP 再用 E1 拆開各自擴充。



/> DU · DISTRIBUTED UNIT

一句話 DU 管「每毫秒都要決定的事」：排程與重傳。

RLC / MAC

即時排程

MAC Scheduler：每個 slot（0.5 / 1 ms）分配 PRB

HARQ 重傳、RLC 分段重組

High-PHY

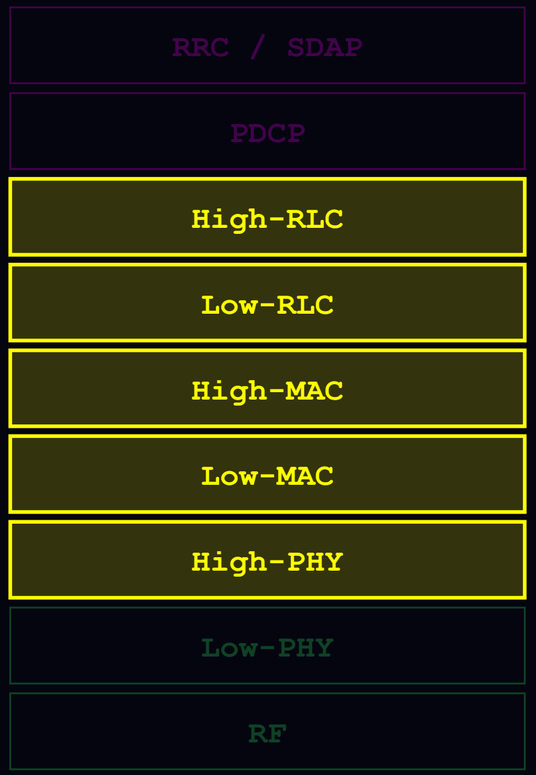
運算最重的一段

LDPC / Polar 編解碼、調變、層對映

常需要 FPGA / GPU / 加速卡

硬即時 → 靠近基地台

延遲預算以微秒計；向上 F1 接 CU，向下前傳接 RU。



/> RU · RADIO UNIT

一句話 RU 就是天線那一端：把數位訊號變成電波。

Low-PHY

數位訊號處理

FFT / iFFT、Cyclic Prefix、PRACH 擷取

數位波束成形（Massive MIMO）

RF

類比前端

DAC / ADC、升降頻、PA / LNA、天線

掛在鐵塔或天花板上

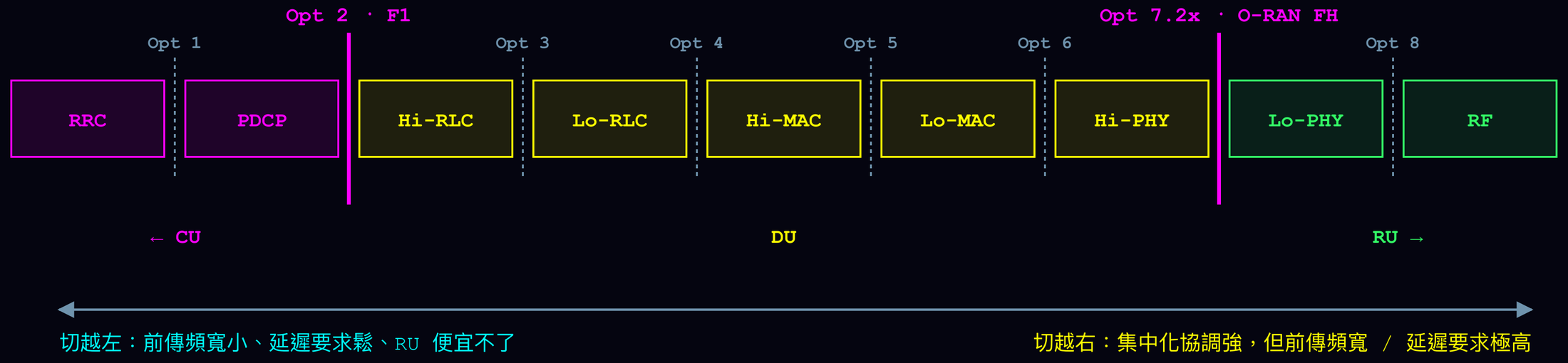
前傳：eCPRI + 精準同步

IQ 資料量大，需 PTP / SyncE 時間同步；O-RAN 7.2x 讓不同廠牌 DU / RU 互通。



FUNCTIONAL SPLIT：要在哪裡切？

一句話 切點越靠近天線越集中、越吃前傳頻寬；業界選 Option 2 和 7.2x。



Option 2 (3GPP F1)

PDCP 以上進 CU；業界標準的 CU / DU 切點。

Option 7.2x (O-RAN)

PHY 中間切開；前傳頻寬與 RU 成本的折衷點。

/> FRONTHAUL / MIDHAUL / BACKHAUL

一句話 越靠近天線的那段網路，越快、越貴、越短。

段落	介面	傳的是什麼	延遲要求
Fronthaul · RU↔DU	eCPRI / O-RAN 7.2x	頻域 IQ 取樣，量最大	~100 µs 等級，距離短
Midhaul · DU↔CU	F1（F1-C / F1-U）	RLC 封包、F1AP 訊令	毫秒等級
Backhaul · CU↔5GC	N2 / N3	NGAP、GTP-U 封包	一般 IP 網路

D-RAN（分散式）
每座站台自帶完整基地台，部署簡單、彼此難協調。

C-RAN（集中式）
DU / CU 集中成資源池，共享算力、易做多站協作。

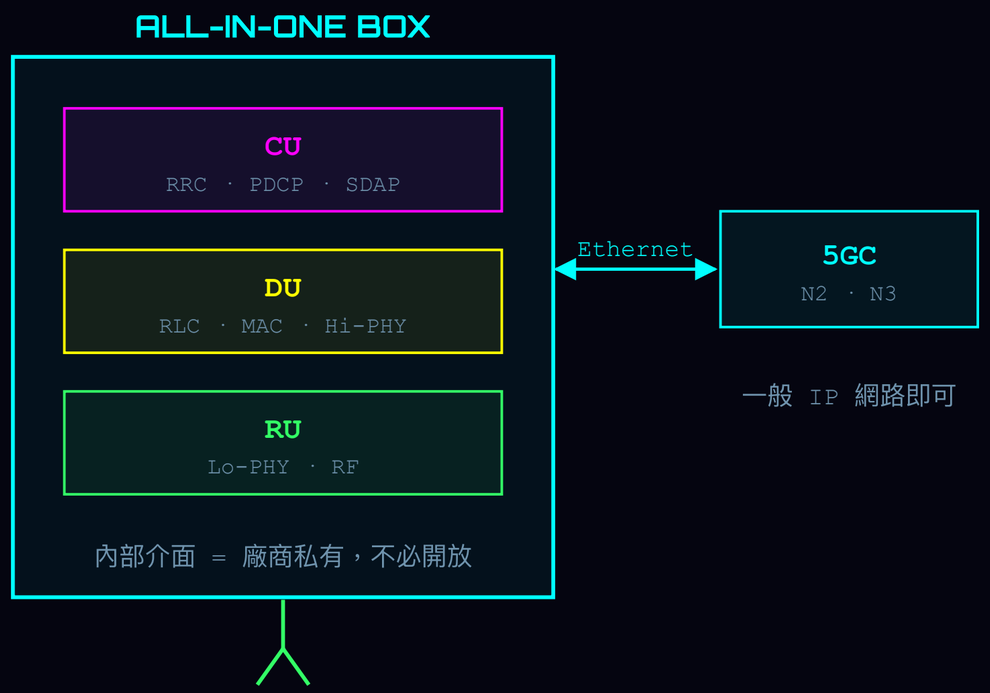
$ PART 1 · RAN TYPE

/> SMALL CELL & O-RAN

同樣是 CU + DU + RU：全部塞進一個盒子，或是拆開、開放介面、讓不同廠商組裝

SMALL CELL：ALL-IN-ONE RAN

一句話 Small Cell = 把 CU、DU、RU 全塞進一個盒子。



特性

低功率 · 小範圍

覆蓋數十到數百公尺

插網路線就能上線，像一台 AP

場景

室內 · 熱點 · 專網

工廠、校園、醫院的企業專網

補強巨站照不到的室內盲區

代價

整合度高、彈性低：單一廠商、難以單獨升級某一層。

/> MACRO CELL VS SMALL CELL

一句話 巨站負責廣、小基站負責密。

面向	MACRO CELL（巨站）	SMALL CELL（小基站）
覆蓋	數公里	數十到數百公尺
發射功率	數十瓦以上	毫瓦到數瓦
架構	常拆成 CU / DU / RU	All-in-one 一體機
部署	鐵塔、機房、前傳光纖	天花板、牆面、一條網路線
典型用途	廣域覆蓋、高速移動	室內補強、熱點容量、企業專網

/> WHAT IS O-RAN？

一句話 O-RAN = 介面開放，讓不同廠商的 RU / DU / CU 可以混搭。

傳統 RAN

RU ═ DU ═ CU 同一家

私有介面，綁定單一廠商

專用硬體，升級要整套換

想加新功能只能等廠商

O-RAN

RU · A 廠 ↔ DU · B 廠 ↔ CU · C 廠

Open：開放介面，多廠商互通

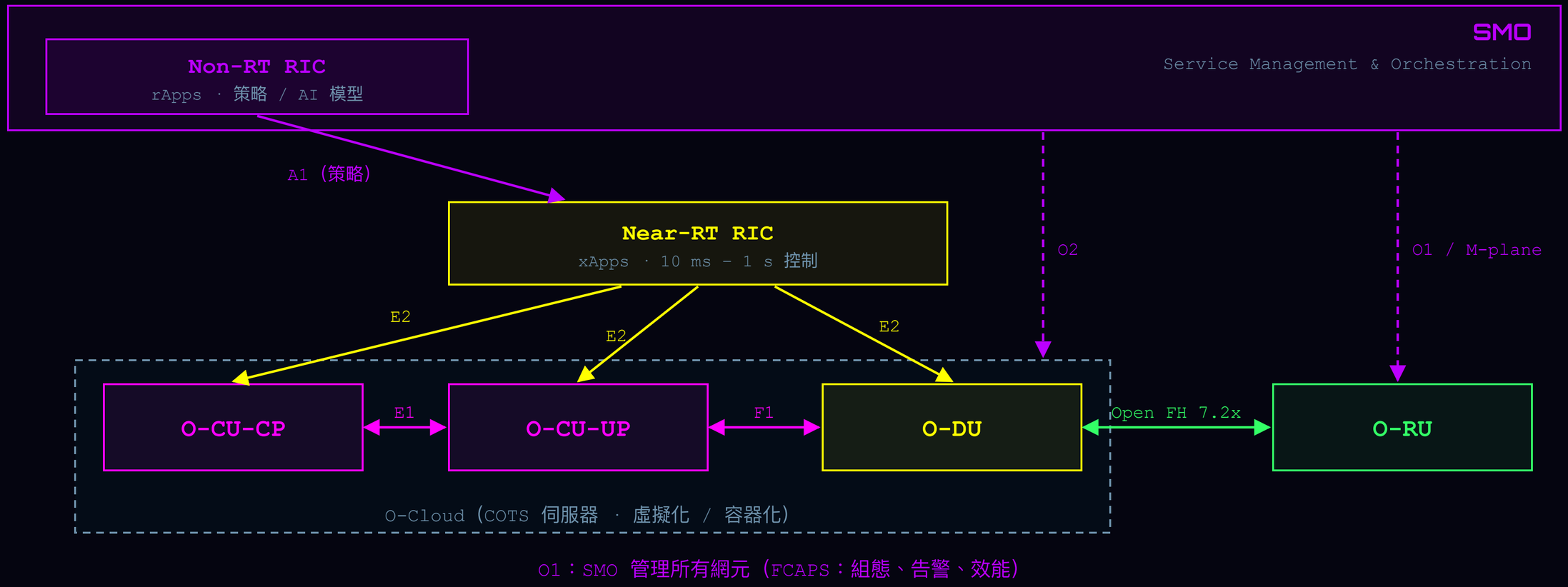
Virtualized：跑在通用伺服器 / 雲上

Intelligent：RIC 讓 AI 參與控制

O-RAN Alliance 成立於 2018（AT&T、中國移動、德國電信、NTT DOCOMO、Orange 發起），在 3GPP 之上補齊開放介面規格。

O-RAN 架構

一句話 在 CU / DU 上面多了兩個「大腦」：Non-RT RIC 與 Near-RT RIC。



/> 三層控制迴圈

一句話 越上層越慢、越聰明；越下層越快、越硬。

Non-RT RIC

> 1 s

- 在 SMO 裡，跑 rApps
- 訓練 AI 模型、下策略
- 經 A1 交給 Near-RT RIC

Near-RT RIC

10 ms – 1 s

- 跑 xApps（可第三方）
- 流量導引、Handover 最佳化
- 經 E2 調整 CU / DU 參數

DU（RT）

< 10 ms

- MAC 排程、HARQ
- 波束管理
- 太快了，只能在 DU 內做

越上層越慢、越「聰明」；越下層越快、越「硬」。

/> OCUDU + RU

一句話 OCUDU 提供開源的 CU / DU 軟體，接上 RU 就是一台真的 gNB。

OCUDU = Open Centralized Unit Distributed Unit

Linux Foundation 下的開源 CU / DU 專案，程式碼承接 srsRAN 5G stack。

目標：carrier-grade 的開源 CU / DU

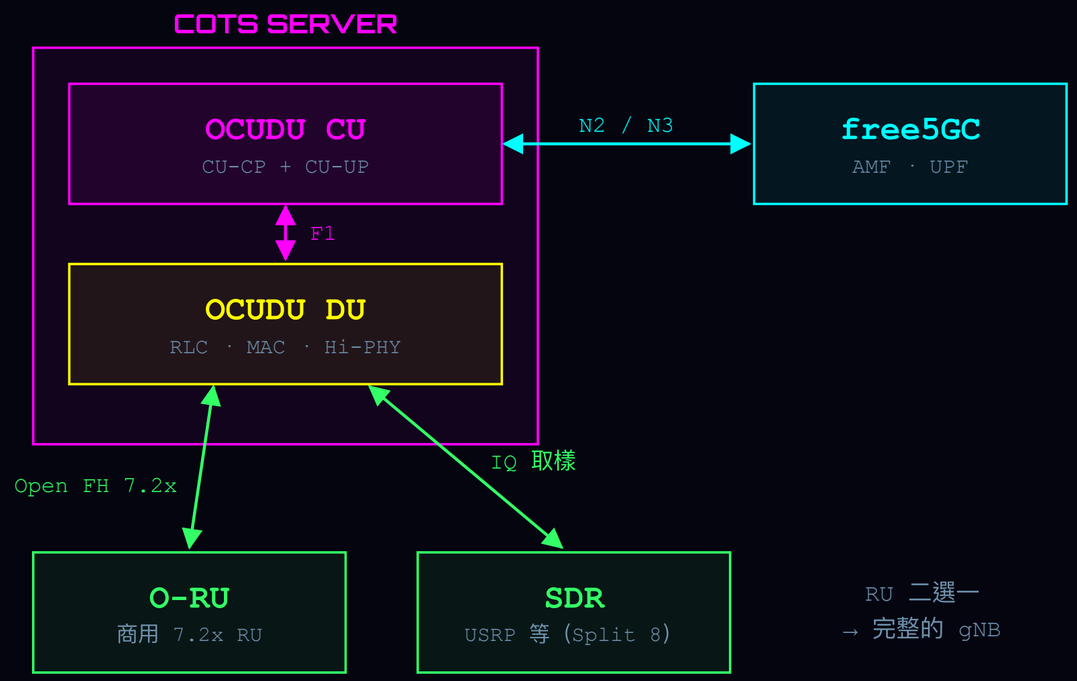
中立治理、寬鬆授權（BSD-3），可跑在 x86、ARM、GPU 平台。

缺的那一塊：RU

OCUDU 只提供軟體；接上 O-RAN 7.2x RU 或 SDR，才是一台會發射的 gNB。

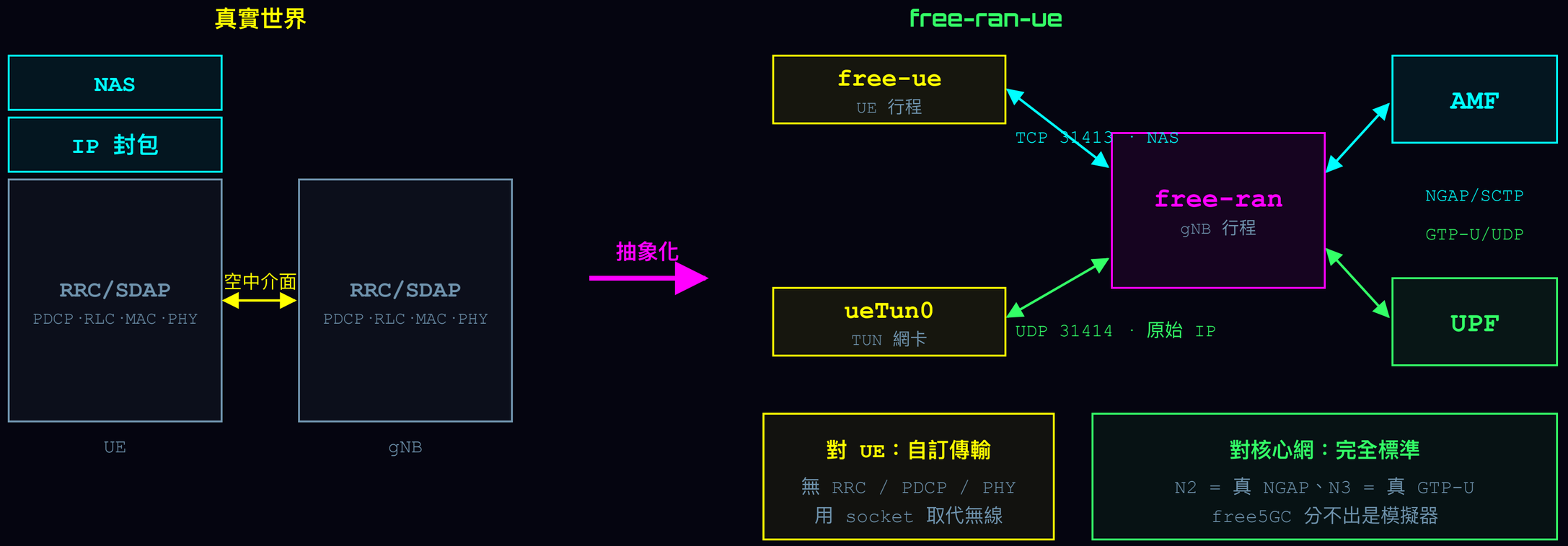
對核心網而言

它就是一個 gNB：同樣用 N2 / N3 接 free5GC。



/> 從真實 RAN 到模擬 RAN

一句話 測核心網不必發射電波：把 Uu 換成 socket，N2 / N3 保持標準。



測核心網，不需要真的發射電波：把 Uu 換成 socket，N2 / N3 保持 100% 標準。

$ PART 2 · FREE-RAN-UE

/> FREE-RAN-UE

為 free5GC 打造的分離式 RAN / UE 模擬器：輕量、可控、可重現，並能測試 NR-DC

/> 為什麼需要 RAN / UE 模擬器？

一句話 模擬器讓測試便宜、可重現、可自動化、可放大規模。

01
成本
商用測試儀器授權昂貴、交期長，按 UE 數或功能模組收費

02
可控、可重現
真實 UE 是黑盒，異常情境難觸發、難精準重現

03
自動化
CI 裡放不進 RF 硬體；模擬器可以每次 commit 都跑

04
規模
一行指令拉起 1,000 個 UE 做併發 / 壓力測試

測核心網時，「穩定地產生 RAN / UE 行為」比「連上真實基地台」更重要。

整合式 VS 分離式

一句話 free-ran-ue 把 gNB 和 UE 拆成兩個行程，更像真實拓樸。

PacketRusher

整合式 · RAN + UE 同一個行程

- 上手快、部署簡單，適合 smoke test
- GTP-U 走 kernel 的 gtp5g 模組
- 要客製特定行為時彈性較小

free-ran-ue

分離式 · gNB / UE 各自獨立行程

- gNB、UE 可分開部署在不同機器 / namespace
- GTP-U 在 user space，好改、好觀察
- 支援 NR-DC、多 UE、Web Console

分離式更貼近真實拓樸：UE 透過「某種連線」接到 gNB，gNB 再接到核心網。

/> FREE-RAN-UE 功能總覽

一句話 一個工具同時給你 gNB、UE、多 UE、NR-DC 和 Web Console。

GNB

free-ran

NGAP over SCTP、userspace GTP-U、Xn

UE

free-ue

5G-AKA、NAS 加密 / 完整性、TUN 網卡

SCALE

ue -n -p

一次起 N 個 UE，最多 P 個同時啟動

NR-DC

Static / Dynamic

建 session 時就雙連線，或事後由 Console 觸發

CONSOLE

Web UI

管理多個 gNB、看 UE 清單、切換 NR-DC

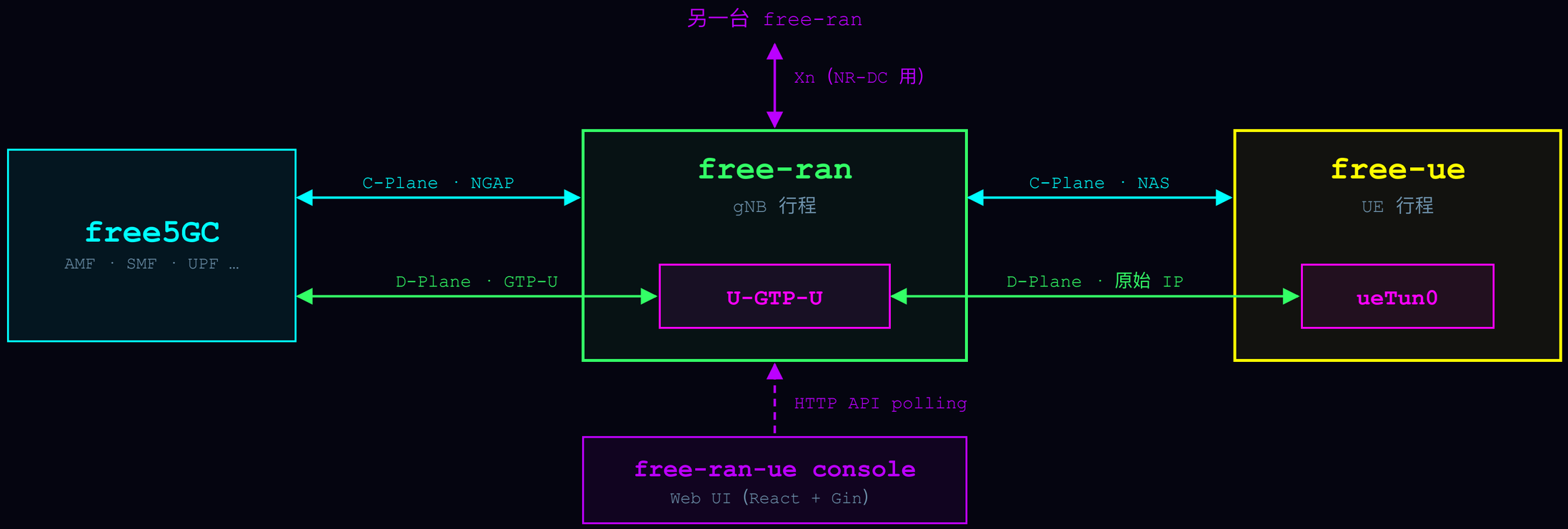
DEPLOY

多種啟動方式

單機、namespace、Docker Compose、Helm

整體架構

一句話 gNB 對外四條基本連線，再加上 Xn 和 Console。



gNB 對外四條基本連線：AMF（NGAP）、UPF（GTP-U）、UE 控制面（NAS）、UE 資料面（IP）。

/> REPO 架構

一句話 一個 binary、三個子指令；gnb/ 與 ue/ 各自獨立，只靠 socket 溝通。

free-ran-ue/

```
.
├── main.go          // 入口
├── cmd/             // gnb / ue / console
├── gnb/             // RAN 本體
│   ├── gnb.go · ngapDispatcher.go
│   ├── gtp.go · xn.go · ranUe.go
├── ue/              // UE 本體
│   ├── ue.go · nas.go · tun.go
├── config/          // 範例設定檔
├── model/ · constant/
└── console/         // Web Console
```

① **cmd：cobra 子指令**

`free-ran-ue gnb` 、 `ue` 、 `console`

② **gnb：今天的主角**

gnb.go 主流程、ngapDispatcher.go 下行、gtp.go 資料面、xn.go NR-DC。

③ **ue：模擬手機**

ue.go 流程、nas.go 訊息、tun.go 網卡。

④ **config：照抄就能跑**

gnb.yaml、ue.yaml，以及 NR-DC 的 static / dynamic 範例。

/> 入口：CMD/GNB.GO

一句話 讀設定 → 建 gNB → Start → 等 Ctrl+C → Stop。

cmd/gnb.go（精簡）

```
func gnbFunc(cmd *cobra.Command, args []string) {
1       util.LoadFromYaml(path, &gnbConfig)
1       util.ValidateGnb(&gnbConfig)

2       gnb := gnb.NewGnb(&gnbConfig, &logger)
3       if err := gnb.Start(ctx); err != nil { return }
        defer gnb.Stop()

4       signal.Notify(sigCh, os.Interrupt, syscall.SIGTERM)
        <-sigCh                          // 等 Ctrl+C
        cancel()
}
```

1 Load + Validate
YAML → model.GnbConfig

2 NewGnb
字串轉 NGAP IE、建 sync.Map

3 Start(ctx)
依序建好所有連線與 goroutine

4 等待訊號
Ctrl+C → cancel → Stop()

/> GNB 設定檔：CONFIG/GNB.YAML

一句話 上半段對核心網、下半段對 UE；IP / Port 要和兩邊對齊。

config/gnb.yaml（節錄）

```yaml
gnb:
  amfN2Ip: "10.0.1.1"
  ranN2Ip: "10.0.1.2"
  upfN3Ip: "10.0.1.1"
  ranN3Ip: "10.0.1.2"
  ranControlPlaneIp: "10.0.2.1"
  ranDataPlaneIp: "10.0.2.1"
  # port：38412 / 2152 / 31413 / 31414
  plmnId: { mcc: "208", mnc: "93" }
  snssai: { sst: "1", sd: "010203" }
  gnbId: "000314"
  api: { ip: "10.0.1.2", port: 40104 }
```

① 對 1 核心網

和 free5GC 的 AMF ngapIpList、UPF N3 位址一致。

② 對 2 UE

gNB 開給 UE 的兩個 port：TCP 傳 NAS、UDP 傳 IP 封包。

③ 身分

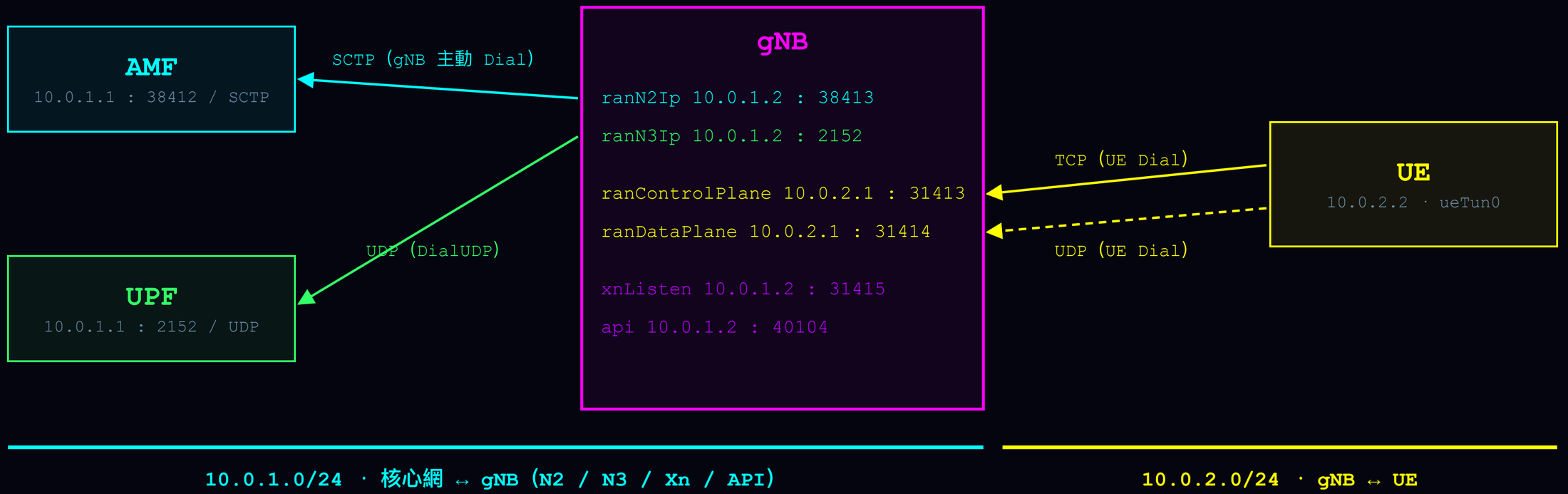
PLMN、TAC、切片要被 AMF 支援，否則 NG Setup 失敗。

④ 給 4 Console

Web Console 會來打這個 API。

/> 位址規劃：兩個網段

一句話 兩個網段：核心網 ↔ gNB 用 10.0.1.x，gNB ↔ UE 用 10.0.2.x。



啟動前先確認兩個網段都可達；卡住時第一優先檢查 IP / Port 是否對齊，再看鑑權與切片。

$ PART 2 · GNB

/> FREE-RAN

Start() 的啟動順序、各 handler 的分佈，以及控制面 / 資料面如何分工與溝通

/> TYPE GNB STRUCT

一句話 gNB = 對外的連線 + 三個 listener + 一堆「UE 查表」。

gnb/gnb.go · 連線（精簡）

```
type Gnb struct {
    amfN2Ip, upfN3Ip, … string
①   n2Conn *sctp.SCTPConn   // → AMF
①   n3Conn *net.UDPConn     // → UPF
    plmnId ie.PLMNIdentity
    tai    ie.TAI
    snssai ie.SNSSAI
    staticNrdc bool
②   ranControlPlaneListener *net.Listener
②   ranDataPlaneServer       *net.UDPConn
②   xnListener               *net.Listener
```

gnb/gnb.go · 狀態（精簡）

```
③   ranUeConns  sync.Map // id → *RanUe
③   xnUeConns   sync.Map // *XnUe → {}
④   dlTeidToUe  sync.Map // teid → Ue
④   addressToUe sync.Map // addr → Ue
    imsiTodlTeidAndUeType sync.Map
    imsiReadyChans        sync.Map
⑤   gtpChannel chan []byte
    teidGenerator *TeidGenerator
    *ngapDispatcher
    api
}
```

① 對 AMF、UPF 的連線

② 等 UE / Xn 的 listener

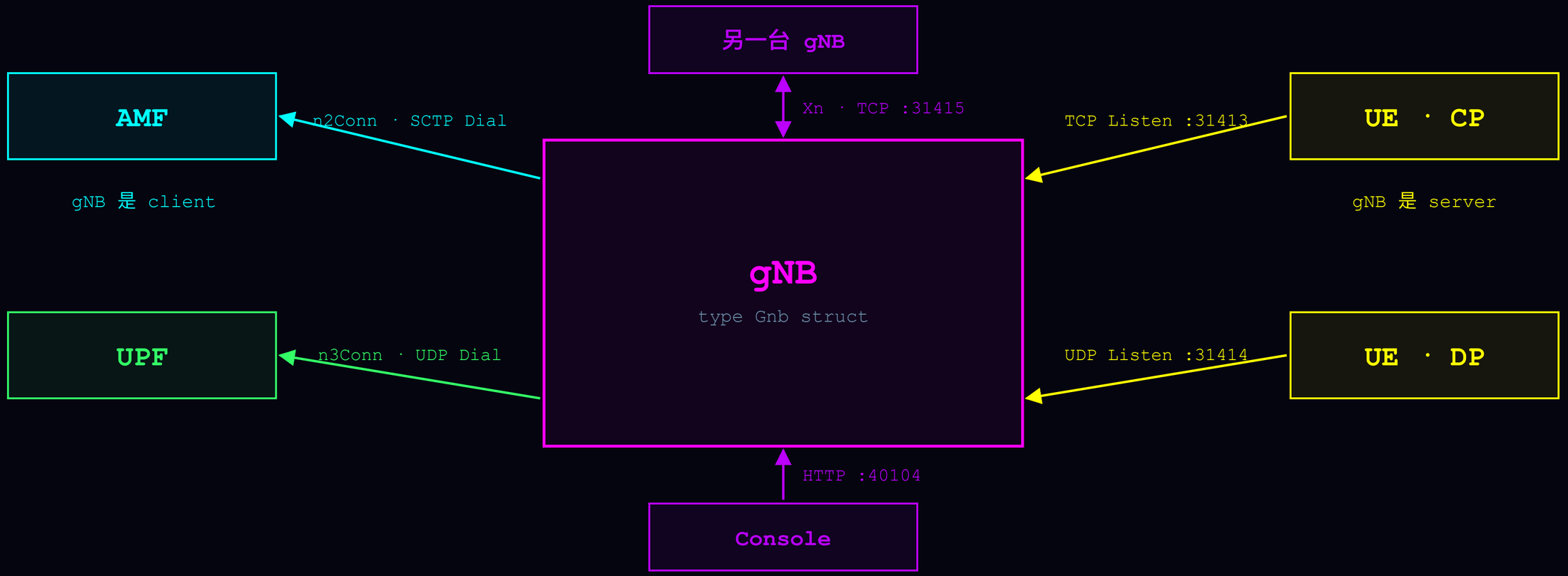
③ UE 清單

④ 資料面查表

⑤ 上行 GTP 封包佇列

/> GNB 的六個 SOCKET

一句話 對核心網是 client，對 UE 是 server。



/> START() 啟動順序

一句話 照順序把 11 件事開起來，前一件成功才做下一件。

01	`connectToAmf()`	SCTP Dial 到 AMF，設定 PPID = 60（NGAP）
02	`setupN2()`	同步送 NG Setup Request、等 Response
03	`connectToUpf()`	UDP Dial 到 UPF :2152
04	`startXnListener()`	若 xnInterface.enable：TCP Listen :31415
05	`go ngapDispatcher.start(g)`	從這裡開始，n2Conn 的讀取交給 dispatcher
06	`startRanControlPlaneListener()`	TCP Listen :31413，等 UE 連進來
07	`startRanDataPlaneServer()`	UDP Listen :31414
08	`startGtpProcessor(ctx)`	兩條 goroutine：UL 寫 N3、DL 讀 N3
09	`go startDataPlaneProcessor()`	讀 UE 送來的 UDP 封包
10	`go Xn / UE accept loop`	接受 Xn 連線、接受 UE 連線
11	`startApiServer()`	Gin HTTP :40104，給 Console 用

/> 為什麼是這個順序？

一句話 先確定核心網接受我，才開門收 UE。

gnb/gnb.go · Start()（精簡）

```
func (g *Gnb) Start(ctx context.Context) error {
    g.connectToAmf()
    g.setupN2()              // NG Setup（同步）
    g.connectToUpf()
    if g.xnInterface.enable { g.startXnListener() }
    go g.ngapDispatcher.start(g)
    g.startRanControlPlaneListener()
    g.startRanDataPlaneServer()
    g.startGtpProcessor(ctx)
    go g.startDataPlaneProcessor()
    go /* Xn、UE accept loop */
    g.startApiServer()
}
```

① 先 N2，才收 UE

AMF 沒接受這台 gNB，收 UE 也沒用。

② setupN2 在 dispatcher 之前

NG Setup 自己 Read；之後 n2Conn 只剩 dispatcher 一個讀者，不會搶訊息。

③ DP server 在 GTP processor 之前

下行 goroutine 需要它把封包寫回 UE。

失敗就逆序關閉

每一步出錯，都會把前面開好的連線關掉。

/> CONNECTTOAMF / CONNECTTOUPF

一句話 gNB 是 client：主動用 SCTP 連 AMF、用 UDP 連 UPF。

gnb/gnb.go（精簡）

```
func (g *Gnb) connectToAmf() error {
    amfAddr, gnbAddr, _ :=
        getAmfAndGnbSctpN2Addr(…)
①   conn, err := sctp.DialSCTP("sctp",
        gnbAddr, amfAddr)
    info, _ := conn.GetDefaultSentParam()
②   info.PPID = constant.NGAP_PPID
    conn.SetDefaultSentParam(info)
    g.n2Conn = conn
    return nil
}
```

gnb/gnb.go（精簡）

```
func (g *Gnb) connectToUpf() error {
    upfAddr, _ := net.ResolveUDPAddr(
        "udp", "10.0.1.1:2152")
    ranAddr, _ := net.ResolveUDPAddr(
        "udp", "10.0.1.2:2152")
③   conn, err := net.DialUDP("udp",
        ranAddr, upfAddr)
    g.n3Conn = conn
    return nil
}
```

① SCTP 四次握手

② PPID = 60，代表 NGAP

③ Connected UDP：固定對端

/> GNB 的 GOROUTINE 地圖

一句話 少數常駐 goroutine + 每個 UE / 訊息 / 封包各自的短命 goroutine。

CONTROL PLANE

go ngapDispatcher.start(g)
讀 n2Conn → 每則 NGAP 再開 go dispatch()

go UE accept loop
Accept TCP :31413 → NewRanUe()

go handleRanConnection(ranUe)
每個 UE 一條：setupN1 → releaseN1

go Xn accept loop
Accept TCP :31415 → go xnInterfaceProcessor

go api.server.ListenAndServe
Gin：/api/gnb/info、/api/gnb/ue/nrdc

DATA PLANE

go startDataPlaneProcessor
ReadFromUDP :31414 (UE → gNB)

go formatGtpPacket…
每個封包：加 GTP header → gtpChannel

go forwardGtpPacketToN3Conn
gtpChannel → n3Conn.Write (→ UPF)

go receiveGtpPacketFromN3Conn
n3Conn.Read → 依 TEID 送回 UE

共享狀態（sync.Map）

ranUeConns ranUeId → *RanUe

xnUeConns *XnUe → {}

dlTeidToUe dlTeid → Ue

addressToUe udpAddr → Ue

imsiTodlTeid… imsi → (teid,type)

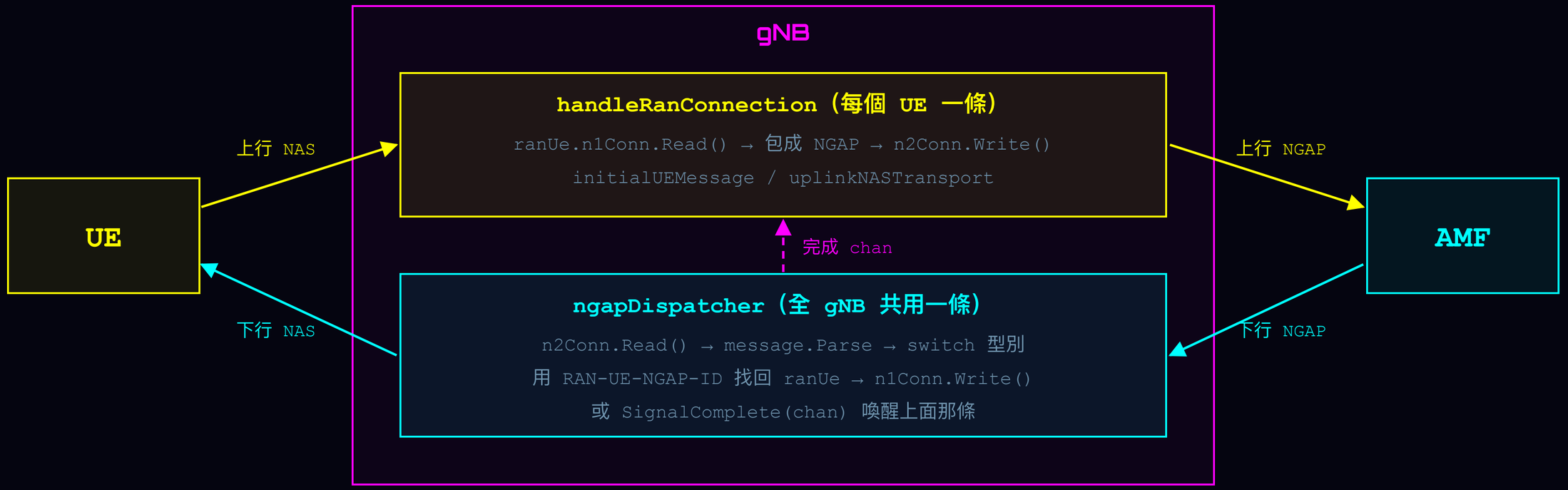
imsiReadyChans imsi → chan

多條 goroutine 併發讀寫
→ 全部用 sync.Map

一個 gNB 行程 = 少數常駐 goroutine + 每個 UE / 每則訊息 / 每個封包各自的短命 goroutine

/> 控制面：上行 / 下行由不同 GOROUTINE 處理

一句話 上行：每個 UE 一條、照順序走；下行：一條 dispatcher 統一分派。



上行：每個 UE 一條、照流程順序讀；下行：AMF 的訊息全部從同一條 SCTP 進來，只能集中分派。

/> UE 連進來之後

一句話 每個 UE 一條 goroutine；這條 goroutine 活多久，UE 就連多久。

gnb/gnb.go · UE accept loop（精簡）

```
  go func() {
    for {
      conn, _ := listener.Accept()
①     ranUe, _ := NewRanUe(conn, idGen)
②     if g.staticNrdc { ranUe.ActivateNrdc() }
③     g.ranUeConns.Store(ranUe.GetRanUeId(), ranUe)
④     go g.handleRanConnection(ctx, ranUe)
    }
  }()
```

① 配一個 RAN-UE-NGAP-ID

②③④ 登記 UE，開專屬 goroutine

⑦ 結束時歸還 ID、TEID

gnb/gnb.go（精簡）

```
  func (g *Gnb) handleRanConnection(ctx, ranUe) {
      defer func() {
⑦         ranUe.GetN1Conn().Close()
⑦         ranUe.Release(idGen, teidGen)
⑦         g.ranUeConns.Delete(ranUe.GetRanUeId())
      }()
⑤     g.setupN1(ranUe)    // 註冊 + PDU Session
⑥     g.releaseN1(ranUe) // 阻塞，等去註冊
  }
```

② Static NR-DC 的伏筆

⑤⑥ 一生：註冊 → 等去註冊

/> SETUPN1：一個 UE 的上行劇本

一句話 註冊 → 配 DL TEID → 建 PDU Session → 登記資料面。

gnb/gnb.go（精簡）

```
func (g *Gnb) setupN1(ranUe *RanUe) error {
①     g.processUeInitialization(ranUe)
②     dlTeid, _ := g.teidGenerator.AllocateTeid()
      ranUe.SetDlTeid(dlTeid)
③     g.processUePduSessionEstablishment(ranUe)
④     g.dlTeidToUe.Store(hex(dlTeid), ranUe)
④     g.imsiTodlTeidAndUeType.Store(imsi,
          dlTeidAndUeType{dlTeid, UE_TYPE_RAN})
⑤     g.markImsiReady(imsi)
      return nil
}
```

① 註冊

Auth、SMC、Registration Complete。

② DL TEID：gNB 自己決定

下行封包是 gNB 收，所以 gNB 配。

③ PDU Session

UL TEID 由核心網給，dispatcher 會解出來。

④⑤ 登記資料面

TEID → UE 查表，並通知「可以綁位址了」。

/> RELAYUPLINKNAS：GNB 只是信差

一句話 讀 UE 的 NAS、不拆開，原封包進 NGAP 送 AMF。

gnb/gnb.go（精簡）

```
func (g *Gnb) relayUplinkNas(ranUe, label) error {
①     n, _ := ranUe.GetN1Conn().Read(buf) // 等 UE
②     msg, _ := getUplinkNasTransport(amfId,
          ranId, g.plmnId, g.tai, buf[:n], g.gnbId)
③     _, err := g.n2Conn.Write(msg)        // 給 AMF
      return err
}

// 註冊期間連續呼叫三次：
④ g.relayUplinkNas(ranUe, "Authentication Response")
④ g.relayUplinkNas(ranUe, "Security Mode Complete")
④ g.relayUplinkNas(ranUe, "Registration Complete")
```

① 阻塞讀 UE

這條 goroutine 就停在這裡等 UE 回答。

② NAS 是不透明的 bytes

不解密、不解析，塞進 NAS-PDU IE。

③ 送給 AMF

帶上兩個 NGAP ID。

④ 中間的下行呢？

Auth Request、SMC 由 dispatcher 寫給 UE。

/> NGAPDISPATCHER：下行 NGAP 分派

一句話 AMF 的訊息全從同一條 SCTP 進來，由一條 dispatcher 讀、依型別分派。

gnb/ngapDispatcher.go（精簡）

```
func (d *ngapDispatcher) start(g *Gnb) {
    buf := make([]byte, 1024)
    for {
①       n, err := g.n2Conn.Read(buf)
        if closed(err) { return }
②       tmp := make([]byte, n)
②       copy(tmp, buf[:n])
③       go d.dispatch(g, tmp)
    }
}
```

gnb/ngapDispatcher.go · dispatch

```
switch msg := ngapMsg.(type) {
④ case *message.DownlinkNASTransport:
④ case *message.InitialContextSetupRequest:
④ case *message.PDUSessionResourceSetupRequest:
④ case *message.UEContextReleaseCommand:
⑤ case *message.PDUSessionResourceModifyConfirm:
default:  // Warn：Unknown NGAP PDU
}
```

① 唯一讀 n2Conn 的 goroutine

② 一定要 copy：buffer 會被下次 Read 覆寫

③ 每則訊息一條 goroutine

④ 四種 UE 流程訊息

⑤ Dynamic NR-DC 專用

/> DOWNLINKNASTRANSPORT：找回是哪個 UE

一句話 靠 RAN-UE-NGAP-ID 找回 UE，再把 NAS 原封寫給它。

gnb/ngapDispatcher.go（精簡）

```
func (d *ngapDispatcher)
        downLinkNASTransportProcessor(g, msg) {
    amfUeNgapId := msg.AMFUENGAPID.Value
①   ranUeNgapId := msg.RANUENGAPID.Value
②   v, ok := g.ranUeConns.Load(ranUeNgapId)
    if !ok { return }
    ranUe := v.(*RanUe)
③   if ranUe.GetAmfUeId() == -1 {
③       ranUe.SetAmfUeId(amfUeNgapId)
    }
④   ranUe.GetN1Conn().Write(msg.NASPDU.Value)
}
```

① RAN-UE-NGAP-ID

gNB 在 NewRanUe 時配發，隨 InitialUEMessage 告訴 AMF。

② 用它找回 UE

同一條 SCTP 上跑著所有 UE 的訊息。

③ 第一次學到 AMF ID

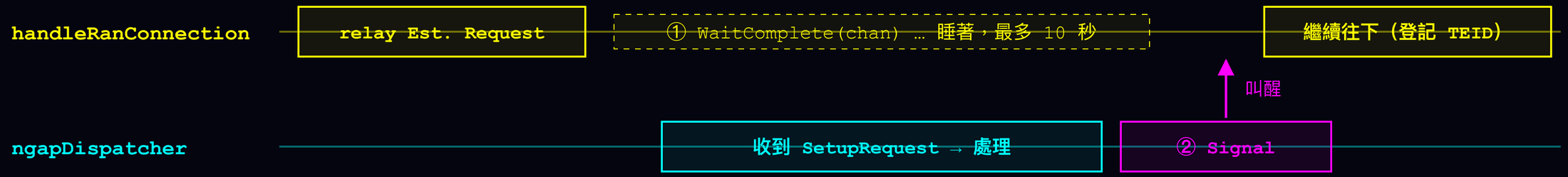
之後的上行都要帶上這對 ID。

④ 原封寫給 UE

走 UE 的控制面 TCP。

/> 兩條 GOROUTINE 怎麼同步？

一句話 上行那條先睡；dispatcher 處理完再用 channel 叫醒它。



等待方 · gnb.go（精簡）

```
   g.relayUplinkNas(ranUe, "PDU Session …")
1  util.WaitComplete(
1      ranUe.GetPduSessionEstablishmentCompleteChan(),
       10*time.Second)
```

通知方 · ngapDispatcher.go（精簡）

```
   g.n2Conn.Write(response)
2  util.SignalComplete(
2      ranUe.GetPduSessionEstablishmentCompleteChan())
```

RanUe 上共三條：PDU Session 建立、去註冊完成（ueContextRelease…）、Dynamic NR-DC 切換（pduSessionModifyIndication…）。

/> RANUE、XNUE 與 UE INTERFACE

一句話 本地 UE 和分流來的 UE 長得不同，但資料面只看共同的 interface。

gnb/ranUe.go（精簡）

```
type RanUe struct {
①    amfUeNgapId int64   // 初始 -1
①    ranUeNgapId int64
     mobileIdentity5GS *ie.MobileId5GS
     ulTeid, dlTeid     []byte
②    n1Conn             net.Conn      // TCP：NAS
②    dataPlaneAddress *net.UDPAddr // UDP：IP
③    pduSessionEstablishmentCompleteChan chan …
     … // 另兩條 completion chan
④    nrdcIndicator     bool
     nrdcIndicatorMtx sync.Mutex
}
```

gnb/xnUe.go

```
type XnUe struct {
⑤    imsi string
     ulTeid, dlTeid    []byte
     dataPlaneAddress *net.UDPAddr
}
```

gnb/ueInterface.go（精簡）

```
⑥ type Ue interface {
     GetIMSI() string
     GetUlTeid() []byte
     GetDlTeid() []byte
     GetDataPlaneAddress() *net.UDPAddr
     SetDataPlaneAddress(*net.UDPAddr)
}
```

① 兩個 NGAP ID

② 控制面 TCP / 資料面 UDP

③ 同步用 chan

④ NR-DC 開關

⑤ XnUe 只認 IMSI

⑥ 資料面只看這個

/> ID 與 TEID 產生器

一句話 從 1 開始找第一個空位；有鎖，所以多個 UE 同時連進來也不會撞號。

gnb/gtp.go · TeidGenerator

```
func (t *TeidGenerator) AllocateTeid() ([]byte, error) {
    t.mtx.Lock()                                          ①
    defer t.mtx.Unlock()
    for i := 1; i <= 65535; i++ {                         ②
        if _, ok := t.teids.Load(int64(i)); !ok {
            t.teids.Store(int64(i), true)
            return hex.DecodeString(                      ③
                fmt.Sprintf("%08x", i))                   ③
        }
    }
    return nil, fmt.Errorf("teid pool exhausted")
}
```

① Mutex

ue -n 100 -p 10 時，很多 UE 同時要 TEID。

② 第一個空位

第一個 UE 的 DL TEID 就是 00000001；釋放後可重用。

③ 4 bytes 的 []byte

直接放進 NGAP 的 GTP-TEID，也直接 copy 進 GTP header。

RanUeNgapIdGenerator 同一招

1-65535 的 RAN-UE-NGAP-ID 池。

/> 資料面：USERSPACE GTP-U

一句話 UE 只送原始 IP；加上 / 拆掉 GTP header 全由 gNB 做。

UPLINK



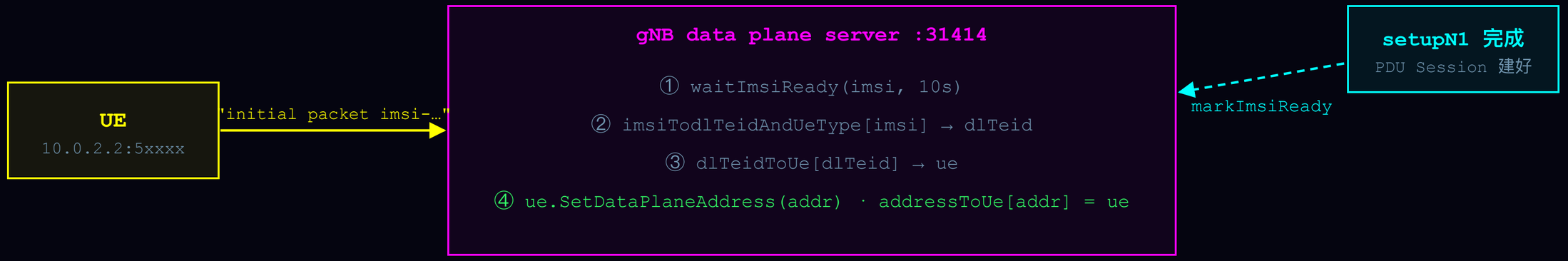
DOWNLINK



UE 完全不碰 GTP；封裝、解封裝、TEID 對應全在 gNB 的 user space 完成（不需要 gtp5g）

/> "INITIAL PACKET"：把 UDP 位址綁到 UE

一句話 UDP 看不出是誰；UE 第一個封包先報上 IMSI，gNB 記住這個位址。



gnb/gnb.go · startDataPlaneProcessor（精簡）

```
n, addr, _ := g.ranDataPlaneServer.ReadFromUDP(buf)
if string(tmp[:len(INIT)]) == INIT {
    imsi := string(tmp[len(INIT)+1 : n])
    go g.handleUeDataPlaneInitialPacket(addr, imsi)
} else {
    go g.handleUeDataPlanePacket(addr, tmp)
}
```

問題

UDP 沒有連線概念，gNB 只看得到來源位址。

解法

第一個封包報 IMSI，之後用 addressToUe 查表。

/> 資料面的四張查表

一句話 四張表，回答同一個問題：「這個封包是誰的？」

MAP	KEY → VALUE	誰寫入	誰查詢
imsiTodlTeidAndUeType	IMSI → (DL TEID, ran/xn)	setupN1 / Xn processor	initial packet
dlTeidToUe	DL TEID → Ue	setupN1 / Xn processor	下行 GTP 封包
addressToUe	UDP 位址 → Ue	initial packet	上行 UE 封包
imsiReadyChans	IMSI → chan struct{}	markImsiReady	waitImsiReady

imsiReadyChans 解決競賽：UE 的 initial packet 可能比 setupN1 收尾更早到，所以要等最多 10 秒。

/> 上行：自己組 GTP-U HEADER

一句話 在 IP 封包前面加 12 bytes，最重要的是 TEID。

① 0x32	② 0xff	③ len+4	④ TEID	00 00 00 00	IP
byte 0 · flags	1 · G-PDU	2-3 · length	4-7 · UL TEID	8-11 · seq / N-PDU / next	payload

gnb/gtp.go（精簡）

```
   gtpHeader := make([]byte, 12)
1  gtpHeader[0] = 0x32 // ver=1, PT=1, S=1
2  gtpHeader[1] = 0xff // G-PDU
3  binary.BigEndian.PutUint16(gtpHeader[2:],
3      uint16(len(packet)+4))
4  copy(gtpHeader[4:], teid)
   // gtpHeader[8:12] 全 0
5  gtpChannel <- append(gtpHeader, packet...)
```

① 0x32 = 0011 0010

Version 1、PT=1、S=1 → header 12 bytes。

③ 不含前 8 bytes

所以是 payload + 4。

⑤ 排進同一條 channel

單一 goroutine 寫到 n3Conn。

/> 下行：依 TEID 找回 UE

一句話 拆掉 GTP header、用 TEID 查表，寫回那個 UE 的 UDP 位址。

gnb/gtp.go · forwardPacketToUe（精簡）

```
① teid, payload, _ := parseGtpPacket(pkt)
② ue, ok := dlTeidToUe.Load(teid)
   if !ok { return }          // 不認識：丟
   addr := ue.(Ue).GetDataPlaneAddress()
③ if addr == nil { return } // 還沒報到
④ server.WriteToUDP(payload, addr)
```

gnb/gtp.go · parseGtpPacket（精簡）

```
   basic, hl := pkt[:8], 8
⑤ if basic[0]&(E|S|PN) != 0 { hl += 3 }
   if basic[0]&E == 0 {
       return teid, pkt[hl:]
   }
   for {  // 跳過 extension header
       switch pkt[hl] {
       case 0x00: return teid, pkt[hl+1:]
⑥      case 0x85: hl += 2 + int(pkt[hl+1])*2
       }
   }
```

① 解析 header

② TEID → UE

③ UE 還沒送 initial packet

④ RanUe、XnUe 都適用

⑤ 有 flag 就多 3 bytes

⑥ PDU Session Container（QFI）

/> API SERVER 與 CONSOLE

一句話 gNB 開兩支 HTTP API；Console 主動來拉資料、或觸發 NR-DC。

gnb/gnb.go · initApiRoutes（精簡）

```
return util.Routes{
    {
1       Method:  http.MethodGet,
1       Pattern: "/info",     // /api/gnb/info
        HandlerFunc: g.handleConsoleGnbInfo,
    },
    {
2       Method:  http.MethodPost,
2       Pattern: "/ue/nrdc", // /api/gnb/ue/nrdc
        HandlerFunc: g.handleConsoleGnbUeNrdcModify,
    },
}
```

① GET /api/gnb/info

gNB 資訊、RanUe 清單（含 NR-DC 狀態）、XnUe 清單。

② POST /api/gnb/ue/nrdc

body：`{"imsi": "imsi-…"}`，切換 Dynamic NR-DC。

Console = pull 模式

新增一台 gNB，只是多一個 API 來源。

$ PART 2 · UE

/> FREE-UE

Start() 的流程、各 handler 的分佈，以及 TUN 網卡與資料面 channel 的管理

/> TYPE UE STRUCT

一句話 UE = 對 gNB 的兩三條連線 + 身分與金鑰 + 一張 TUN 網卡。

ue/ue.go（精簡）

```
type Ue struct {
①    ranControlPlaneConn net.Conn // TCP · NAS
①    ranDataPlaneConn    net.Conn // UDP · IP
①    dcRanDataPlaneConn  net.Conn // UDP · NR-DC
②    authentication     // supi、kAmf、secCtx
③    authenticationSubscription // K、OPc、SQN
     pduSession          // dnn、sNssai
④    nrdc                // enable、specifiedFlow
⑤    ueTunnelDevice *water.Interface
     readFromTun, readFromRan chan []byte
     pduSessionEstablishmentAccept // ueIp
}
```

① 兩條（DC 時三條）
控制面 TCP；資料面 UDP，DC 再多一條。

② secCtx
NAS 金鑰、COUNT、演算法。

③ 訂閱資料
要和 Webconsole 完全一致。

④ nrdc
DC 開關與分流規則。

⑤ TUN 裝置
songgao/water 建立。

/> UE START() 啟動順序

一句話 先控制面：註冊、建 session 拿到 IP；再開資料面與 TUN。

ue/ue.go · Start()（精簡）

```
func (u *Ue) Start(ctx, wg) error {
1       u.connectToRanControlPlane()
2       u.processUeRegistration()
3       u.processPduSessionEstablishment()
4       u.connectToRanDataPlane()
5       u.setupTunnelDevice()
6       go u.waitForRanMessage(ctx, wg)
7       go u.handleDataPlane(ctx, wg)
        return nil
}
```

1 **控制面先通**
之後每則 NAS 都走這條 TCP

2·3 **同步一問一答**
readAndDecodeNas 阻塞等回應

4 **拿到 IP 才開資料面**
送 initial packet 報 IMSI

5 **TUN 綁 UE IP**
例：10.60.0.1/32

6·7 **兩條常駐 goroutine**
控制面一條、資料面一條

/> 多 UE：UE -N 1000 -P 10

一句話 同一份設定複製 N 份，MSIN 和網卡名稱自動 +1，最多 P 個同時啟動。

cmd/ue.go（精簡）

```
1  sem := util.NewSemaphore(maxConcurrent) // -p
   for i := 0; i < num; i++ {                // -n
       go func(index int) {
1          sem.Acquire(); defer sem.Release()
           cfg := ueConfig
2          updateUeConfig(&cfg, baseMsin, baseTun, index, …)
           u := ue.NewUe(&cfg, &logger)
           u.Start(ctx, &wg)
       }(i)
   }
3  cfg.Ue.Msin = fmt.Sprintf("%0*d", msinLen, baseMsin+num)
4  cfg.Ue.UeTunnelDevice = fmt.Sprintf("%s%d", baseTun, num)
```

① Semaphore 限流

避免 1,000 個 UE 同時註冊把 AMF 打爆。

② 每個 UE 一份設定

複製後再改兩個欄位。

③ MSIN 遞增

Webconsole 要先批次建好這些訂閱。

④ ueTun0、ueTun1 …

每個 UE 一張網卡；需要 root。

/> TUN 裝置：KERNEL 與 USER SPACE 的橋

一句話 建一張虛擬網卡；kernel 送進來的封包，程式讀得到、也寫得回去。

ue/tun.go（精簡）

```
tunCfg := water.Config{DeviceType: water.TUN}
tunCfg.Name = name                      // ueTun0
tun, _ := water.New(tunCfg)            // /dev/net/tun
cmds := [][]string{
    {"ip", "addr", "add", ip + "/32", "dev", name},
    {"ip", "link", "set", "dev", name, "up"},
}
for _, c := range cmds {
    exec.Command(c[0], c[1:]...).Run()
}
```

① TUN = L3 虛擬網卡

程式 Read 到的是完整 IP 封包。

② /32、沒有預設路由

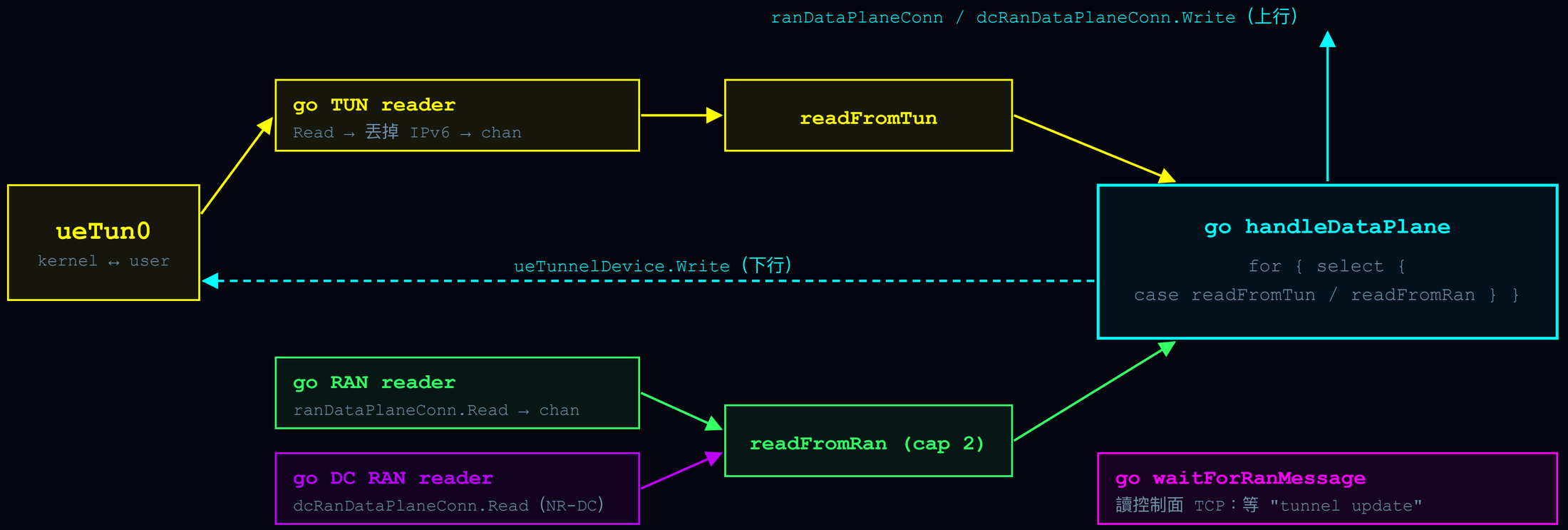
所以要指定網卡：`ping -I ueTun0 8.8.8.8`

③ 網卡 up

ignoreSetupTunnel 可跳過（只測控制面時）。

UE 資料面：GOROUTINE + CHANNEL

一句話 三個讀者把封包丟進 channel，中間一條 goroutine 負責轉送。



/> HANDLEDATAPLANE：SELECT 迴圈

一句話 一條 goroutine 兩邊搬：TUN → gNB（上行）、gNB → TUN（下行）。

ue/ue.go（精簡）

```
select {
① case buf := <-u.readFromTun:                  // 上行
      if !u.isNrdcEnabled() {
          u.ranDataPlaneConn.Write(buf)
②     } else if util.IsIpInSpecifiedFlow(buf, flow) {
          u.dcRanDataPlaneConn.Write(buf)
      } else {
          u.ranDataPlaneConn.Write(buf)
      }
③ case buf := <-u.readFromRan:                  // 下行
      u.ueTunnelDevice.Write(buf)
}
```

① 上行：預設走 Master
沒開 NR-DC 時只有一條路。

② NR-DC 才分流
依目的 IP 選連線 —— 第七章細看。

③ 下行不分來源
兩台 gNB 的封包都寫回 TUN。

/> UE STOP()：收拾順序

一句話 先跟網路說再見，再拆網卡、關資料面，最後才關控制面。

ue/ue.go（精簡）

```
func (u *Ue) Stop() {
    u.processUeDeregistration()
    u.cleanUpTunnelDevice()
    u.ranDataPlaneConn.Close()
    if u.isNrdcEnabled() {
        u.dcRanDataPlaneConn.Close()
    }
    u.ranControlPlaneConn.Close()
}
```

① 去註冊

在這之前 cmd 先 cancel ctx，waitForRanMessage 讓出 TCP。

② 拆 TUN

ip link down / delete。

④ 控制面最後關

去註冊的 NAS 還要走它。

/> UE 設定檔：CONFIG/UE.YAML

一句話 連哪台 gNB、我是誰、我的金鑰 -- 要和 gNB 與 Webconsole 一致。

config/ue.yaml（節錄）

```yaml
ue:
  ranControlPlaneIp: "10.0.2.1"
  ranDataPlaneIp: "10.0.2.1"
  plmnId: { mcc: "208", mnc: "93" }
  msin: "0000000001"
  authenticationSubscription:
    encPermanentKey: "8baf473f2f8f…"
    encOpcKey: "8e27b6af0e69…"
    sequenceNumber: "000000000023"
  integrityAlgorithm: { nia2: true }
  cipheringAlgorithm: { nea0: true }
  pduSession: { dnn: "internet", … }
```

① 連哪台 gNB

等於 gNB 開給 UE 的位址與 port。

② 我是誰

SUPI = imsi-208930000000001。

③ K / OPc / SQN

不對就 Authentication Failure。

④ NIA2 + NEA0

有完整性、不加密，方便 Wireshark 看。

$ PART 3 · PROCEDURE

/> PROCEDURE

NG Setup → Registration → PDU Session Establishment → Deregistration：把 sequence diagram 對到程式碼

/> 四個流程在哪裡發生？

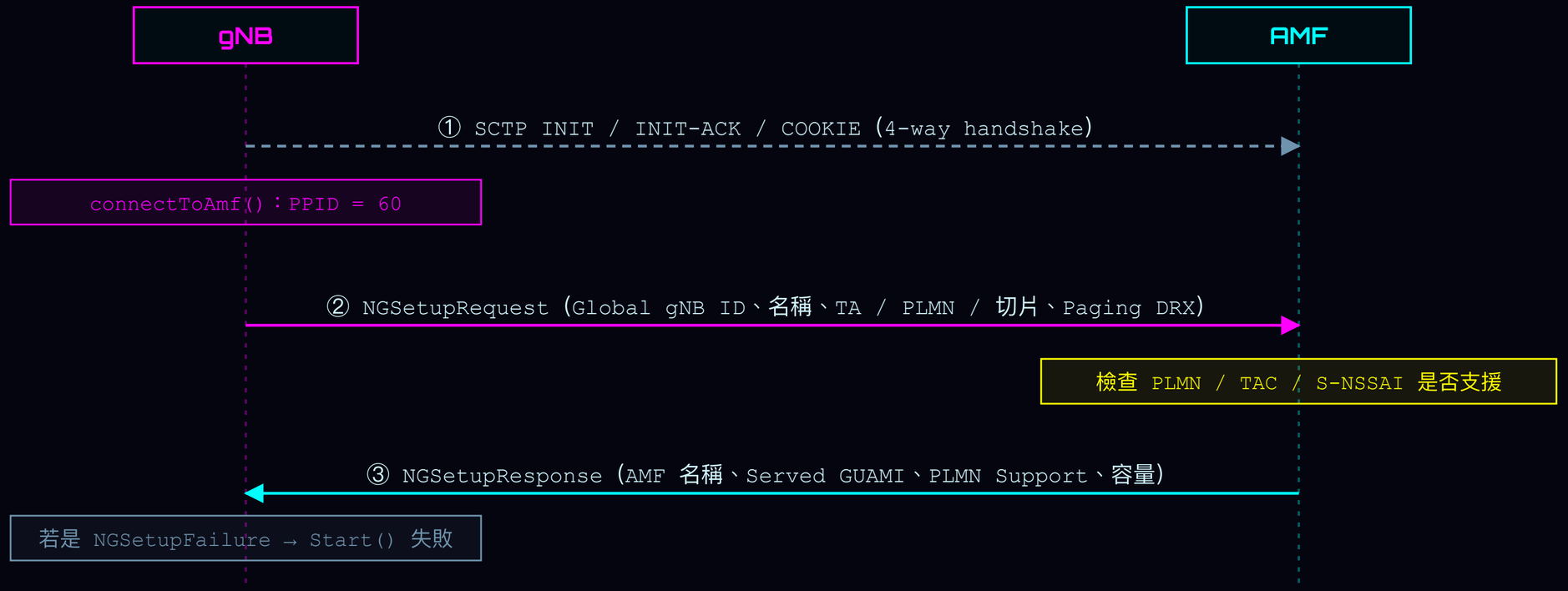
一句話 每個流程都要同時看三個地方：UE、gNB 上行、gNB dispatcher。

流程	觸發時機	GNB 端	UE 端
NG Setup	gNB 啟動	`setupN2()`	—
Registration	UE 連上 gNB	`processUeInitialization()`	`processUeRegistration()`
PDU Session	註冊完成後	`processUePduSession…()`	`processPduSession…()`
Deregistration	UE 收到 Ctrl+C	`processUeDeRegistration()`	`processUeDeregistration()`

每個流程都要同時看三個地方：UE 的同步函式、gNB 的上行函式、gNB 的 dispatcher。

NG SETUP

一句話 gNB 開機先向 AMF 報到：「我是誰、我服務哪些地區和切片」。



這是 gNB 與 AMF 之間「非 UE 相關」的流程：先確認彼此支援的 PLMN、TA、切片，之後才能服務 UE。

/> NG SETUP：程式碼

一句話 組一則 NGSetupRequest 送出，然後同步等 Response。

gnb/ngapBuilder.go（精簡）

```
   &message.NGSetupRequest{
①      GlobalRANNodeID: … GlobalGNBID{
           PLMNIdentity: &plmnId,
           GNBID: BitString{gnbId, len*8} },
②      RANNodeName: gnbName,
③      SupportedTAList: … {
           TAC: tai.TAC,
           BroadcastPLMNList: … {
             TAISliceSupportList: {&snssai} } },
       DefaultPagingDRX: V128,
   }
```

gnb/gnb.go · setupN2（精簡）

```
④  g.n2Conn.Write(request)
⑤  n, _ := g.n2Conn.Read(responseRaw)
   resp, _ := message.Parse(responseRaw[:n])
⑥  _, ok := resp.(*message.NGSetupResponse)
   if !ok {
       return fmt.Errorf("error NGAP setup")
   }
   g.NgapLog.Infof("gNB ID: %s …", …)
```

① gNB ID 22-32 bits

② gNB 名稱

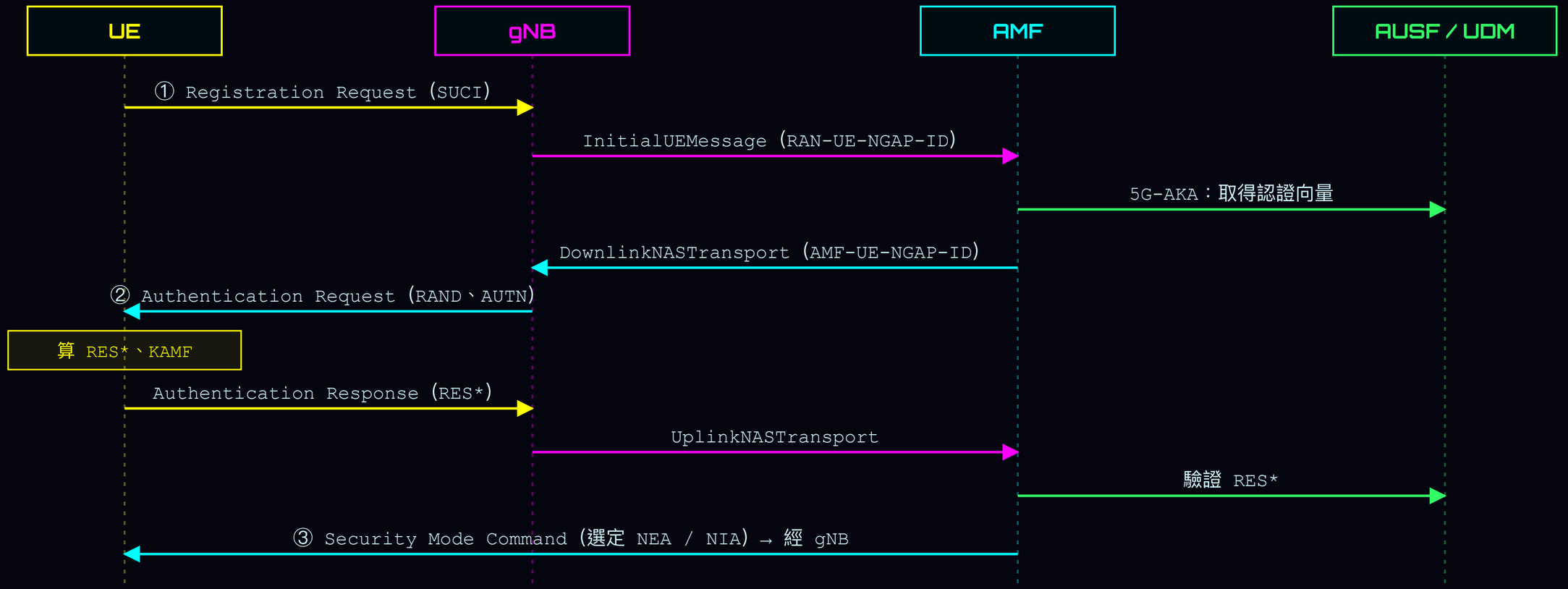
③ 支援的 TA / PLMN / 切片

④⑤ 同步一問一答

⑥ Failure → Start 失敗

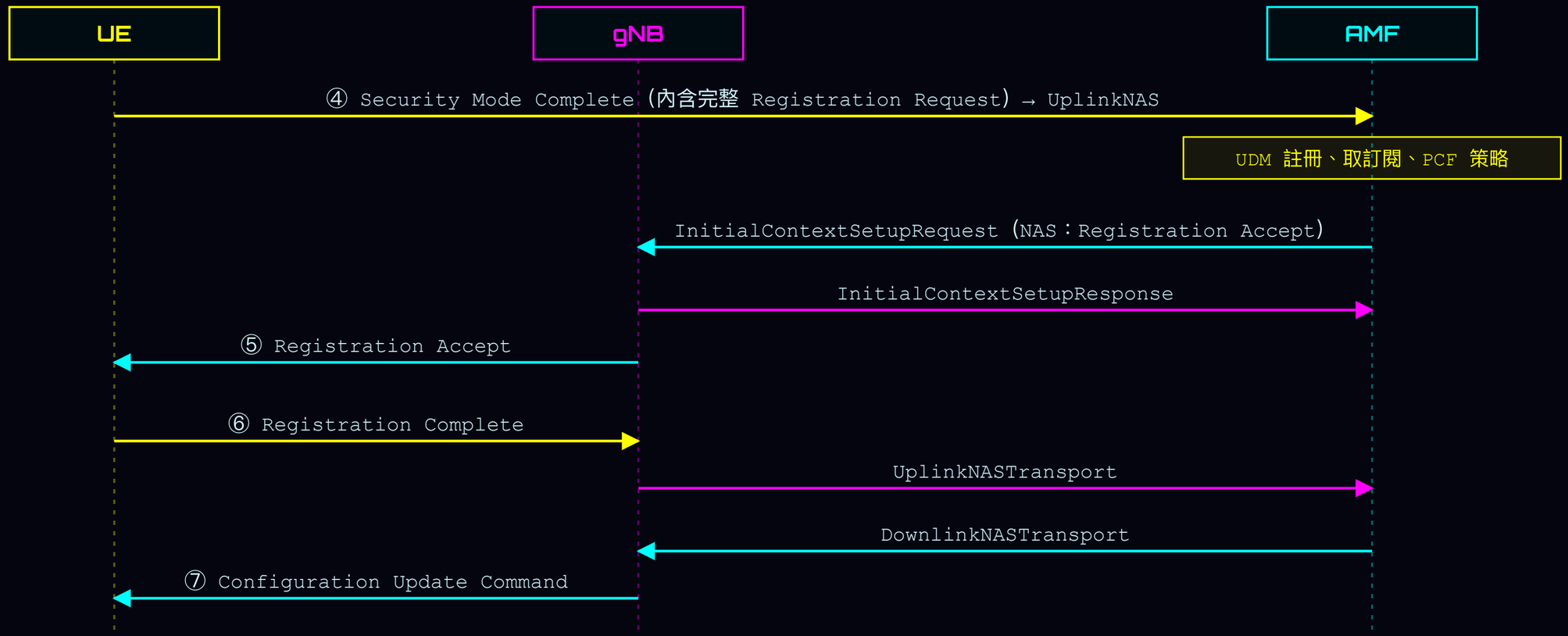
/> REGISTRATION（1/2）：認證

一句話 先證明身分：AMF 出題（RAND），UE 用金鑰算出答案（RES*）。



/> REGISTRATION（2/2）：安全與接受

一句話 身分確認後，開啟加密保護，AMF 正式接受註冊。



/> REGISTRATION：GNB 端

一句話 實線是這條 goroutine 做的上行；註解是 dispatcher 同時在做的下行。

gnb/gnb.go · processUeInitialization（精簡）

```
1  n, _ := ranUe.GetN1Conn().Read(regReq)
1  gmm, _ := nasMessage.ParseGMM(regReq[:n])
   ranUe.SetMobileIdentity5GS(gmm.(*RegReq).MobileId5GS)
2  msg, _ := getInitialUeMessage(ranUeId, regReq[:n], …)
   g.n2Conn.Write(msg)
   // dispatcher：Auth Request → UE
3  g.relayUplinkNas(ranUe, "Authentication Response")
   // dispatcher：Security Mode Command → UE
3  g.relayUplinkNas(ranUe, "Security Mode Complete")
   // dispatcher：InitialContextSetupRequest
3  g.relayUplinkNas(ranUe, "Registration Complete")
   // dispatcher：Configuration Update → UE
```

① SUCI → IMSI

唯一一次解析 NAS：為了知道是誰。

② InitialUEMessage

唯一不帶 AMF-UE-NGAP-ID 的訊息（AMF 還沒配）。

③ 三次「等 UE 回答」

上行一條、下行一條，平行完成。

/> INITIALCONTEXTSETUPREQUEST

一句話 沒有真的 RRC，所以直接回 Response，再把 Registration Accept 轉給 UE。

gnb/ngapDispatcher.go（精簡）

```
func (d *ngapDispatcher)
        initialContextSetupProcessor(g, msg) {
    nasPdu := clone(msg.NASPDU.Value) // Reg Accept
    ranUe := g.ranUeConns.Load(ranUeNgapId)
1   if ranUe.GetAmfUeId() != amfUeNgapId { return }
2   resp, _ := getNgapInitialContextSetupResponse(
        amfUeNgapId, ranUeNgapId)
2   g.n2Conn.Write(resp)              // 先回 AMF
3   ranUe.GetN1Conn().Write(nasPdu) // 再給 UE
}
```

① ID 一致性檢查

每個 dispatcher handler 都會比對，防止送錯 UE。

② 真實 gNB 會更忙

推導 AS 金鑰、啟動 RRC 安全 —— 模擬器省略。

③ Accept 給 UE

UE 收到才會回 Registration Complete。

/> REGISTRATION：UE 端認證

一句話 用 K、OPc 和網路給的 RAND 算出 RES*，證明「我是我」。

ue/ue.go · processUeRegistration（精簡）

```
1  u.ranControlPlaneConn.Write(registrationRequest)
2  authReq, _ := readAndDecodeNas[*message.AuthReq](u, …)
   rand := authReq.AuthParamRAND5GAuthChlg.Rand
   autn := authReq.AuthParamAUTN5GAuthChlg.Autn
   snName := "5G:mnc093.mcc208.3gppnetwork.org"
3  kAmf, kenc, kint, resStar, newSqn, _ :=
3      deriveResStarAndSetKey(supi, …, K, OPc,
3                             rand, autn, snName)
   // 存 KnasEnc / KnasInt、更新 SQN
4  resp, _ := getAuthenticationResponse(resStar)
4  u.ranControlPlaneConn.Write(resp)
```

① Registration Request
帶 SUCI。

② readAndDecodeNas[T]
Go 泛型：讀 → 解 → 斷言型別。

③ Milenage
驗 AUTN，算出 RES* 與 KAMF。

④ 回答
AMF / AUSF 比對 RES*。

/> REGISTRATION：UE 端 SMC 與完成

一句話 照 AMF 選的演算法重算金鑰，之後每則 NAS 都受保護。

ue/ue.go · processUeRegistration（精簡）

```
1  smc, _ := readAndDecodeNas[*message.SecModeCmd](u, …)
1  algos := smc.SelectedNASSecAlgos
1  u.secCtx.CipheringAlg = algos.CipheringAlgo
1  u.secCtx.IntegrityAlg = algos.MsgIntAlgo
   deriveAlgorithmKey(u.kAmf, …)        // 重算金鑰
2  complete := getNasSecurityModeCompleteMessage(reg)
2  enc, _ := nasEncode(complete, u.secCtx,
2      …CipheredWithNew5gNasSecCtx)
   u.ranControlPlaneConn.Write(enc)
   readAndDecodeNas[*message.RegAccept](u, …)
3  u.ranControlPlaneConn.Write(registrationComplete)
4  readAndDecodeNas[*message.CfgUpdateCmd](u, …)
```

① 以 AMF 的選擇為準

UE 只宣告能力，SMC 決定用哪組。

② 第一則受保護 NAS

夾帶完整的 Registration Request。

③ Registration Complete

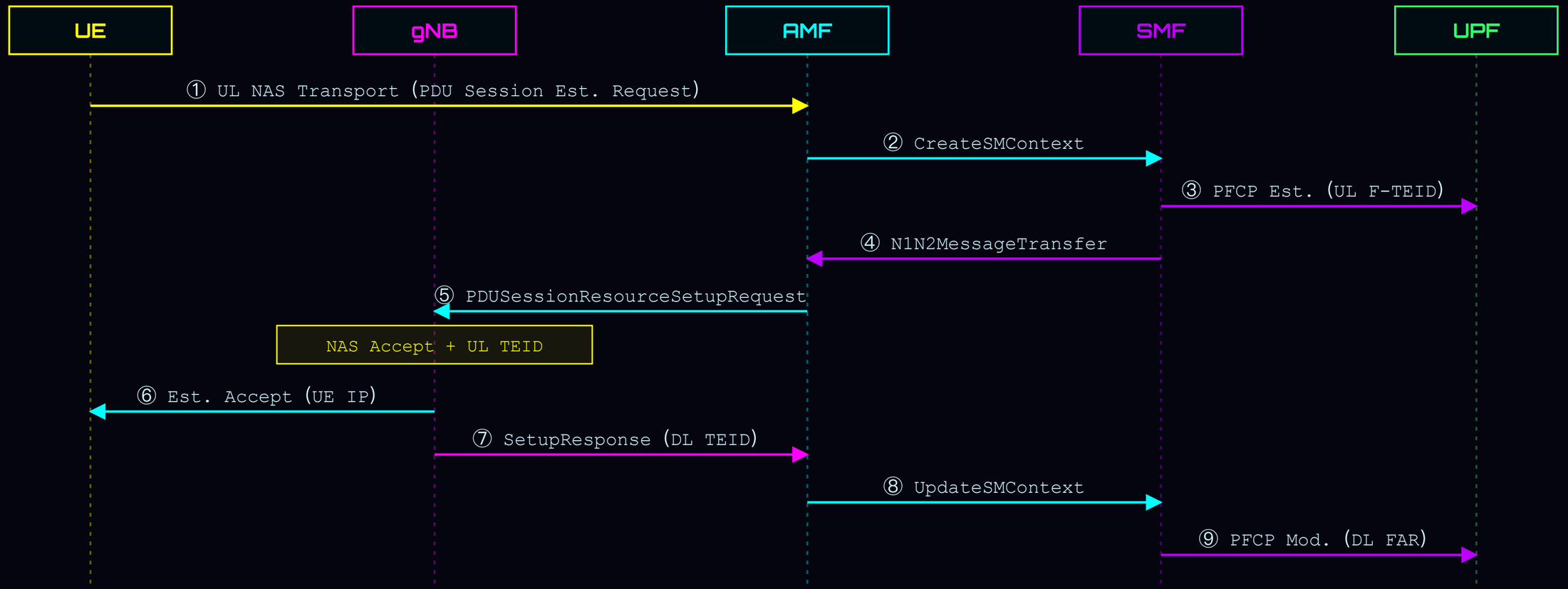
收到 Accept 之後回覆。

④ Config Update

最後一則，收到才算註冊結束。

/> PDU SESSION ESTABLISHMENT

一句話 UE 要上網：核心網配 IP、建隧道，gNB 回報下行要送哪裡。



/> PDU SESSION：GNB DISPATCHER

一句話 拿出 UL TEID、把 Accept 給 UE、回報自己的 DL TEID。

gnb/ngapDispatcher.go（精簡）

```
1  ie.UnmarshalBinary(*item.…RequestTransfer, &transfer)
1  gtp := ULNGUUPTNLInformation.Choice.(*ie.GTPTunnel)
1  ranUe.SetUlTeid(gtp.GTPTEID.Value)          // UL TEID
2  if ranUe.IsNrdcActivated() {
2      qosItem, _ = g.xnPduSessionResource…(imsi, raw)
   }
3  ranUe.GetN1Conn().Write(nasPdu)             // → UE
4  rt, _ := getPduSessionResourceSetupResponseTransfer(
4      ranUe.GetDlTeid(), g.ranN3Ip, 1, g.staticNrdc, qosItem)
5  g.n2Conn.Write(response)                     // → AMF
5  util.SignalComplete(…EstablishmentCompleteChan())
```

① Transfer 是包中包

SMF 產生、AMF 不拆，要再解一次。

② NR-DC 分支

先記住，第七章回來看。

③ Accept 給 UE

裡面有 UE IP 與 QoS Rules。

④⑤ 回報 DL TEID

然後叫醒等待中的 goroutine。

/> SETUPRESPONSETRANSFER：回報 DL TEID

一句話 告訴核心網：「下行請送到 10.0.1.2、TEID = 我配的那個」。

gnb/ngapBuilder.go（精簡）

```
transfer := ie.PDUSessionResourceSetupResponseTransfer{
    DLQosFlowPerTNLInformation: … {
        UPTransportLayerInformation: … GTPTunnel{
            GTPTEID: dlTeid,
            TransportLayerAddress: ranN3Ip },
        AssociatedQosFlowList: … {QFI: qosId},
    },
}
if nrdcIndicator && 有拿到 Secondary 的 TEID {
    transfer.AdditionalDLQosFlowPerTNLInformation =
        &ie.QosFlowPerTNLInformationList{{qosItem}}
}
```

① DL QoS Flow per TNL
下行隧道的目的地。

② TEID + 位址
UPF 下行封包會帶這個 TEID。

③ QFI = 1
預設 QoS Flow。

④ Additional DL…
第二條下行隧道 —— NR-DC 的關鍵。

/> PDU SESSION：UE 端

一句話 送出請求，從 Accept 裡拿到自己的 IP 與 QoS 規則。

ue/ue.go（精簡）

```
1 ul := getUlNasTransportMessage(estReq, PDU_SESSION_ID,
1     ie.ReqType_InitialReq, dnn, sNssai)
  u.ranControlPlaneConn.Write(nasEncode(ul, …))
  dl, _ := readAndDecodeNas[*message.DLNASTransport](u, …)
  gsm, _ := getNasPdu…EstablishmentAccept(dl)
  switch accept := gsm.(type) {
  case *message.PDUSessEstAccept:
2     u.ueIp = ipv4(accept.PDUAddr.IPv4)
3     rules, _ := accept.AuthoQosRules.MarshalBinary()
3     u.nrdc.specifiedFlow = util.GetQosRule(rules, …)
  case *message.PDUSessEstRej:
      return fmt.Errorf("not implemented")
  }
```

① SM 訊息包在 MM 裡

外層 UL NAS Transport 讓 AMF 轉給 SMF。

② UE IP

之後設定到 ueTun0。

③ QoS Rules → specifiedFlow

packet filter（如 1.1.1.1/32）—— NR-DC 分流依據。

TEID 是怎麼交換的？

一句話 誰收封包，誰決定 TEID。

規則：「收的那一端」決定 TEID —— 封包上帶的永遠是接收方給的 TEID

一條 PDU Session（單連線）= 1 個 UL TEID + 1 個 DL TEID



DL TEID：gNB 自己配發（teidGenerator）

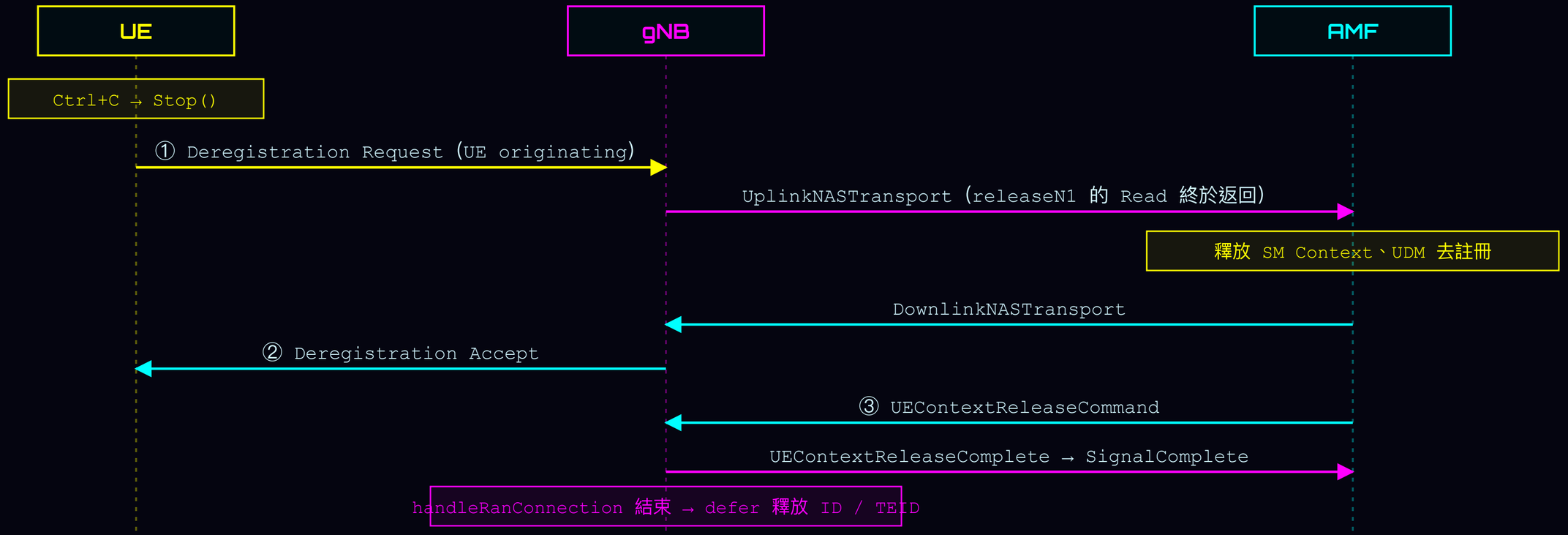
PDUSessionResourceSetupResponseTransfer
.DLQosFlowPerTNLInformation → SMF → UPF FAR

UL TEID：UPF / SMF 配發

PDUSessionResourceSetupRequestTransfer
.ULNGUUPTNLInformation → ranUe.SetUlTeid()

/> DEREGISTRATION

一句話 UE 說再見，AMF 回應，gNB 把這個 UE 的資源全部收回。



/> DEREGISTRATION：程式碼

一句話 UE 送出請求等 Accept；gNB 收到 Release Command 回 Complete，然後收工。

ue/ue.go（精簡）

```
① dereg := getUeDeRegistrationRequest(
①     ie.AccessType_3gpp, 0x00, // switchOff
      ie.NASKeyNA, mobileIdentity5GS)
  enc, _ := nasEncode(dereg, u.secCtx, …)
  u.ranControlPlaneConn.Write(enc)
② readAndDecodeNas[*message.DeregAcceptUEOrig](u, …)
```

gnb/ngapDispatcher.go（精簡）

```
  func (d *ngapDispatcher)
          ueContextReleaseProcessor(g, msg) {
③     pair := msg.UENGAPIDs.Choice.(*UENGAPIDPair)
      ranUe := g.ranUeConns.Load(pair.RANUENGAPID…)
④     g.n2Conn.Write(releaseComplete)
⑤     util.SignalComplete(
          ranUe.GetUeContextReleaseCompleteChan())
  }
```

① switchOff = 0：要等 Accept

② 收到才算去註冊完成

③ 用 ID pair 找 UE

④ 回 Release Complete

⑤ 叫醒 releaseN1

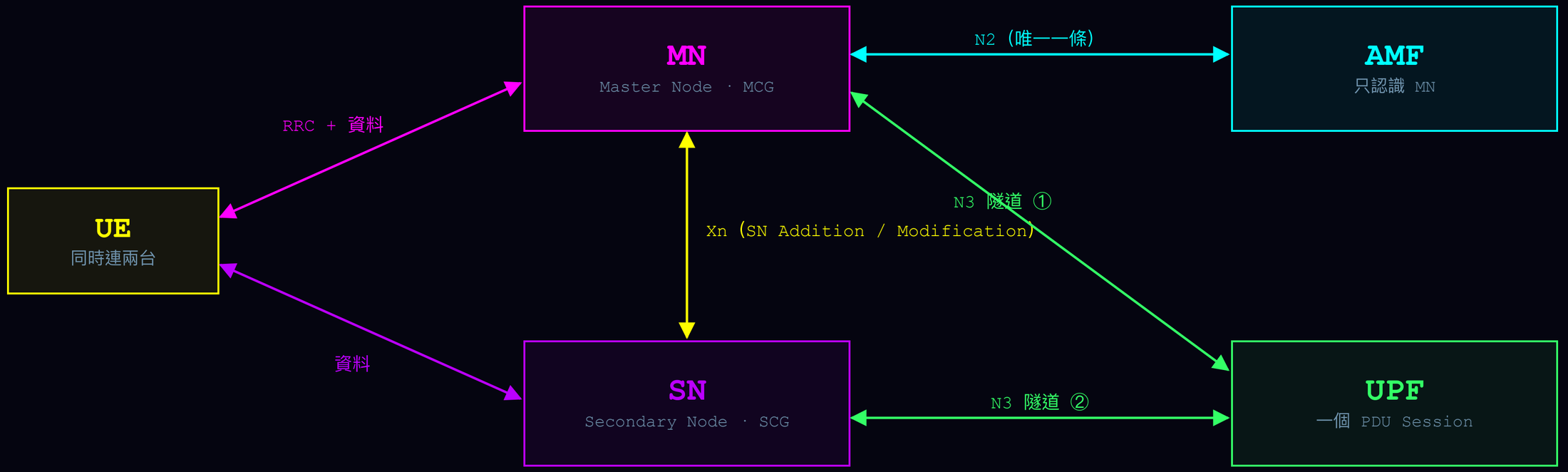
$ PART 3 · NR-DC

/> NR-DC

New Radio Dual Connectivity：一個 UE、兩台 gNB、一個 PDU Session 裡的兩條 N3 隧道 —— 逐行看 free-ran-ue 怎麼做

/> 什麼是 DUAL CONNECTIVITY？

一句話 一個 UE 同時連兩台基地台：控制面只走 Master，資料可以分兩路。



好處：吞吐量疊加、可靠度、負載分攤；控制面仍然只有 MN 一個錨點，核心網看到的還是「一個 UE」。

/> MR-DC 家族

一句話 NR-DC = 兩台都是 5G 基地台、接 5GC 的雙連線。

名稱	MASTER NODE	SECONDARY NODE	核心網
EN-DC	LTE eNB	NR en-gNB	EPC（NSA Option 3x）
NGEN-DC	ng-eNB	gNB	5GC
NE-DC	gNB	ng-eNB	5GC
NR-DC	gNB	gNB	5GC ← 今天的主題

MCG / SCG

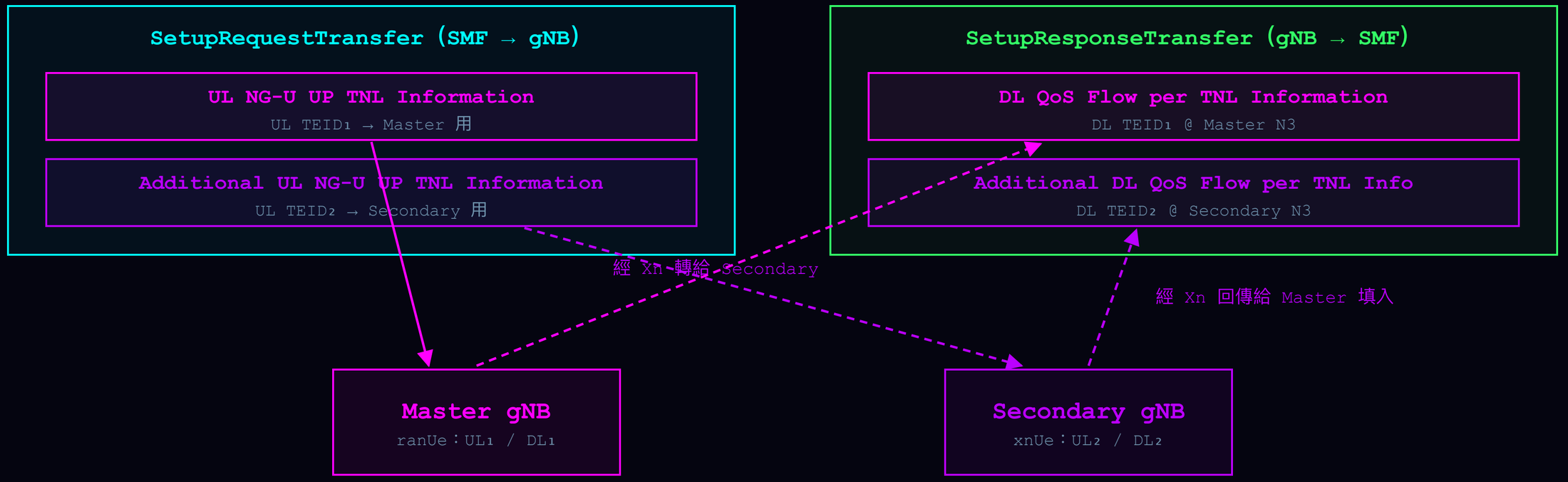
Master / Secondary Cell Group：兩台基地台各自的服務小區群組。

SN Addition / Modification

3GPP 用 XnAP 訊息在 MN、SN 之間協商資源。

/> 核心網眼中的 NR-DC：兩條 N3

一句話 同一個 PDU Session 有兩組 TEID；帶「Additional」的那組屬於 Secondary。



一個 PDU Session、兩組 TEID；UPF 依規則決定某些流量走第二條隧道

FREE5GC 端需要什麼？

一句話 核心網要配合：SMF 給第二組 TEID、UPF 依規則分流。

SMF 支援 NR-DC

在 SetupRequestTransfer 帶 Additional UL NG-U UP TNL Information，並處理 Additional DL 回覆。

Webconsole：Flow Rule

訂閱者至少要有一條 flow rule（例：1.1.1.1/32），它就是要分流到 SN 的流量。

UPF：兩組 FAR

依 PDR 匹配結果，把特定流量的下行封包送到第二條隧道。

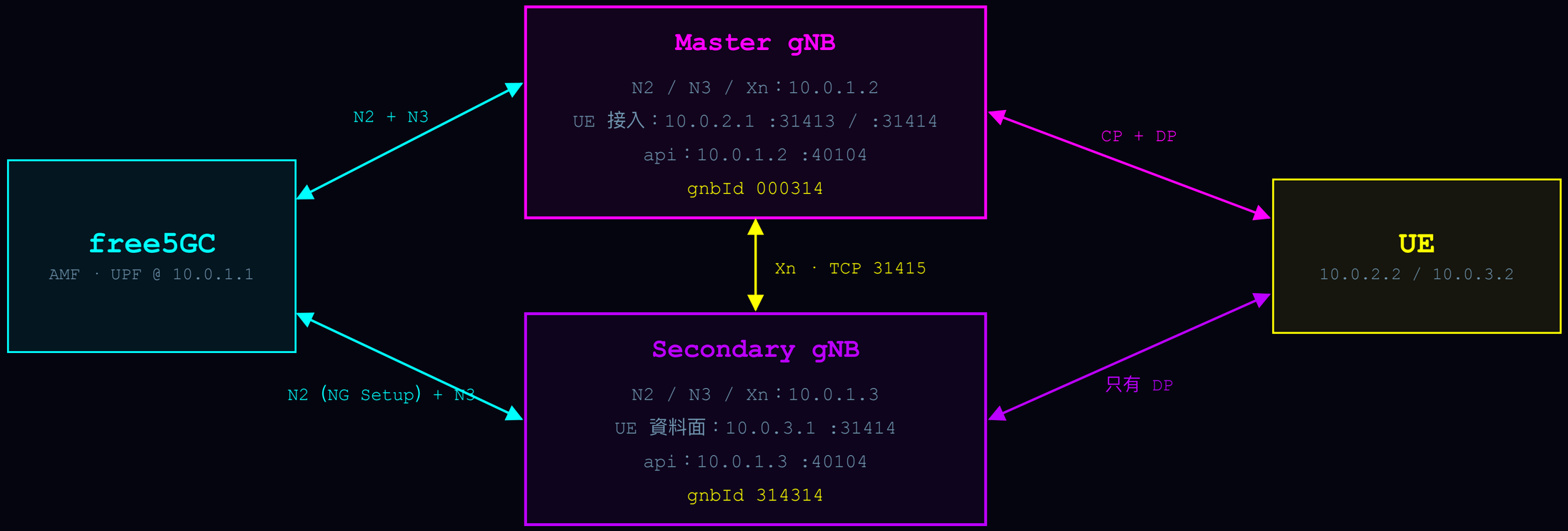
參考文件

- free5GC NR-DC Design Document
- free5gc.org/blog/20250219
- free-ran-ue 設計文件：Static / Dynamic NR-DC

核心網的實作細節屬於 SMF / UPF，本週聚焦在 RAN / UE 端怎麼配合。

/> FREE-RAN-UE 的 NR-DC 拓樸

一句話 UE 連兩台 gNB；兩台 gNB 之間有 Xn。



/> NR-DC 設定檔

一句話 兩台 gNB 互相指向對方的 Xn；UE 多一段 nrdc 指向 Secondary。

gnb-dc-static-master

```
   ranN3Ip: "10.0.1.2"
   ranDataPlaneIp: "10.0.2.1"
   gnbId: "000314"
1  staticNrdc: true
   xnInterface:
2    enable: true
     xnListenIp: "10.0.1.2"
3    xnDialIp: "10.0.1.3"
```

gnb-dc-static-secondary

```
   ranN3Ip: "10.0.1.3"
   ranDataPlaneIp: "10.0.3.1"
   gnbId: "314314"
1  staticNrdc: true
   xnInterface:
2    enable: true
     xnListenIp: "10.0.1.3"
3    xnDialIp: "10.0.1.2"
```

ue-dc-static

```
   ranDataPlaneIp: "10.0.2.1"
   localDataPlaneIp: "10.0.2.2"
   nrdc:
4    enable: true
     dcRanDataPlane:
5      ip: "10.0.3.1"
       port: 31414
5    dcLocalDataPlaneIp: …3.2
```

① Static 開關（Dynamic 版為 false）

②③ Xn：自己 listen、dial 對方

④ UE 一開始就雙連線（Dynamic 版為 false）

⑤ Secondary 的資料面、UE 的第二個來源 IP

/> STATIC VS DYNAMIC NR-DC

一句話 Static：建 session 時就雙連線；Dynamic：事後隨時切換。

面向	STATIC	DYNAMIC
何時建立第二條隧道	PDU Session 建立的同時	Session 建好之後，隨時切換
NGAP 程序	PDUSessionResourceSetup	PDUSessionResourceModifyIndication / Confirm
觸發者	設定檔 staticNrdc: true	Console → POST /api/gnb/ue/nrdc
Xn 往返次數	1 次	開：2 次（Indication + Confirm）；關：1 次
UE 何時連 SN	connectToRanDataPlane()	收到 "tunnel update" → updateDataPlane()

/> FREE-RAN-UE 的 XN：極簡設計

一句話 free-ran-ue 的 Xn 只做一件事：在兩台 gNB 之間交換 TEID。

① ImsiLength	② Imsi	③ Data
2 bytes · BigEndian	ASCII "imsi-2089300…"	原始 NGAP PDU（或 IE bytes）

不是 3GPP XnAP

不實作 SN Addition 等 XnAP 訊息，目的只有一個：交換 TEID。

Payload 直接用 NGAP

Master 把 AMF 給的 NGAP 原封不動轉過去，Secondary 自己解析。

TCP、一問一答

每次請求：Dial → Write → 等 5 秒 Read → Close，沒有長連線。

IMSI 是唯一鑰匙

Secondary 沒有 NGAP ID，只能用 IMSI 找到對應的 XnUe。

/> XNPDU：MARSHAL / UNMARSHAL

一句話 先寫長度、再寫 IMSI、最後接 NGAP —— 收的人照順序切開。

① ImsiLength	② Imsi	③ Data
2 bytes · BigEndian	ASCII "imsi-2089300…"	原始 NGAP PDU（或 IE bytes）

gnb/xn.go（精簡）

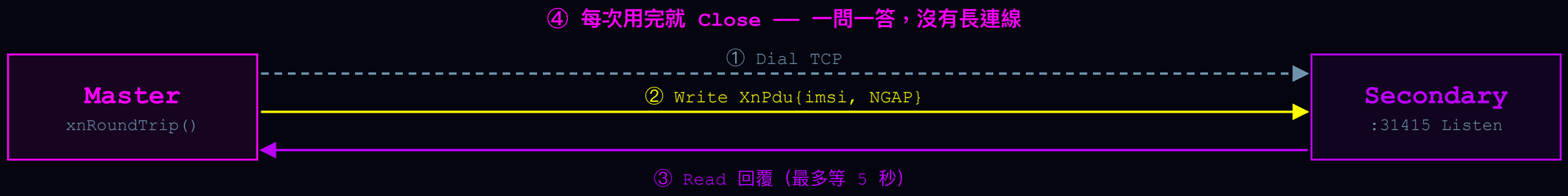
```
func (x *XnPdu) Marshal() ([]byte, error) {
    imsiBytes := []byte(x.Imsi)
1   buffer := make([]byte, 2)
1   binary.BigEndian.PutUint16(buffer,
1       uint16(len(imsiBytes)))
2   buffer = append(buffer, imsiBytes...)
3   buffer = append(buffer, x.Data...)
    return buffer, nil
}
```

gnb/xn.go（精簡）

```
func (x *XnPdu) Unmarshal(data []byte) error {
    if len(data) < 2 { return errShort }
1   x.ImsiLength = BigEndian.Uint16(data[:2])
    data = data[2:]
    if len(data) < int(x.ImsiLength) { … }
2   x.Imsi = string(data[:x.ImsiLength])
3   x.Data = data[x.ImsiLength:]
    return nil
}
```

/> XNROUNDTRIP：MASTER 端的 XN RPC

一句話 像打一通電話：撥號、說完、等回答、掛斷。



gnb/xn.go（精簡）

```
func (g *Gnb) xnRoundTrip(imsi string, payload []byte)
        (respData []byte, err error) {
1   xnConn, err := util.TcpDialWithOptionalLocalAddress(
        g.xnInterface.xnDialIp, g.xnInterface.xnDialPort, "")
4   defer xnConn.Close()  // close 失敗也會回報
2   xnConn.Write(NewXnPdu(imsi, payload).Marshal())
3   xnConn.SetReadDeadline(time.Now().Add(5 * time.Second))
    n, err := xnConn.Read(buffer)
    respPdu.Unmarshal(buffer[:n])
    return respPdu.Data, nil
}
```

三個呼叫者

SetupRequestTransfer、ModifyIndication、ModifyConfirm。

③ 5 秒逾時

Secondary 沒回應時，Master 不會永遠卡住。

/> XN 分派：XNINTERFACEPROCESSOR

一句話 Secondary 看「轉過來的是哪種 NGAP」，就知道要做哪件事。

gnb/xn.go（精簡）

```
func xnInterfaceProcessor(conn net.Conn, g *Gnb) {
    n, _ := conn.Read(buffer)
①   xnPdu.Unmarshal(buffer[:n])
①   ngapMsg, _ := message.Parse(xnPdu.Data)
    switch msg := ngapMsg.(type) {
②   case *message.PDUSessionResourceSetupRequest:
        xnPduSessionResourceSetupProcessor(…)
③   case *message.PDUSessionResourceModifyIndication:
        xnPduSessionResourceModifyIndicationProcessor(…)
④   case *message.PDUSessionResourceModifyConfirm:
        xnPduSessionResourceModifyConfirmProcessor(…)
    }
}
```

① 拆信封

XnPdu ⟶ IMSI + NGAP。

② Static

建 session 時就加入。

③ Dynamic 第一步

開：附加 $TEID_2$；關：拆掉 XnUe。

④ Dynamic 第二步

補上 UL $TEID_2$。

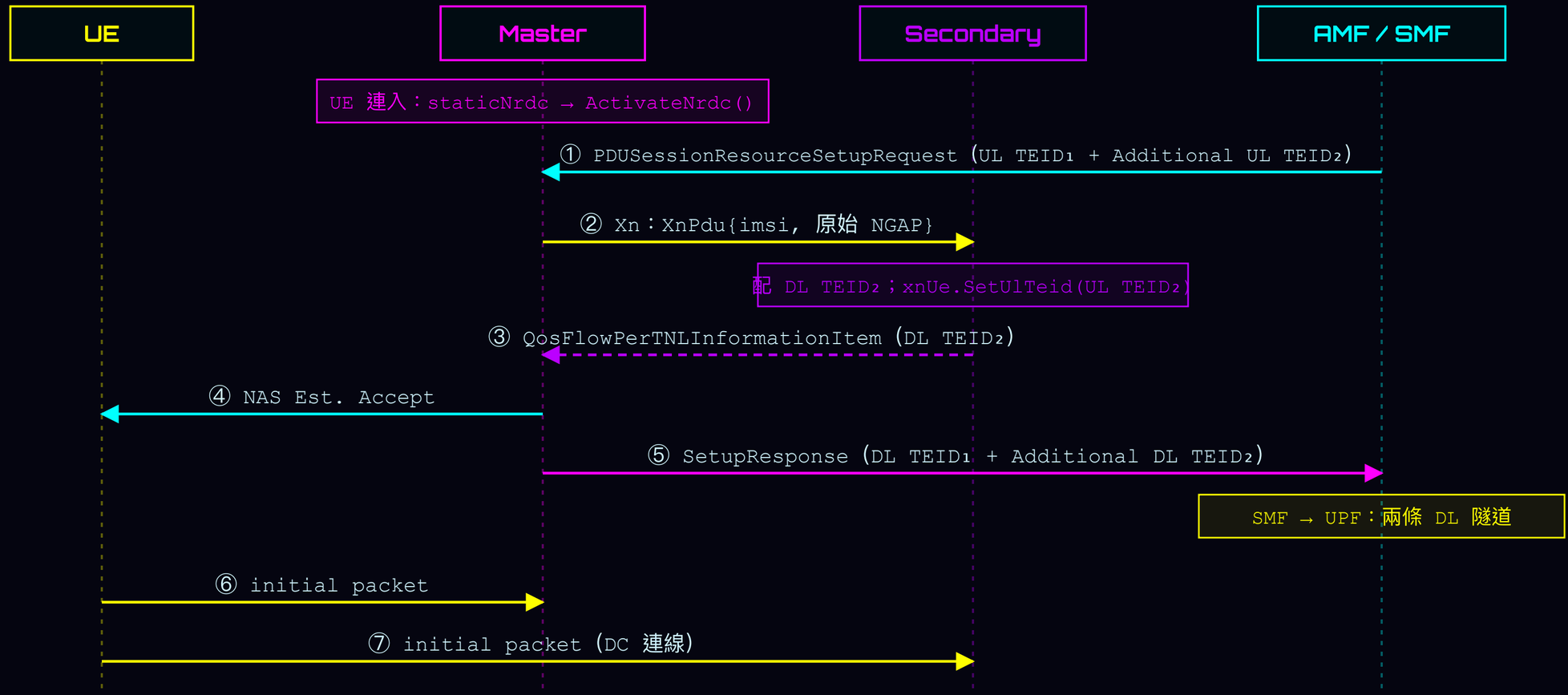
$ NR-DC · STATIC

/> STATIC

在 PDU Session Establishment 的同時，就把第二條隧道建好

/> STATIC NR-DC：完整流程

一句話 建 session 時 Master 問 Secondary 要 TEID，一起回報給核心網。



/> STATIC ①②：MASTER 轉送 SETUP REQUEST

一句話 Master 收到 Setup Request，先把整則 NGAP 轉給 Secondary 問 TEID。

gnb/ngapDispatcher.go（精簡）

```
① if ranUe.IsNrdcActivated() {
②     qosItem, err = g.xnPduSessionResource…Transfer(
          ranUe.GetMobileIdentityIMSI(), ngapRaw)
  }
```

gnb/gnb.go（精簡）

```
  func (g *Gnb) xnPduSessionResource…Transfer(
          imsi string, raw []byte) (item …, err error) {
③     respData, err := g.xnRoundTrip(imsi, raw)
      if err != nil { return item, err }
④     ie.UnmarshalBinary(respData, &item)
      return item, nil
  }
```

① Static：一連進來就啟用

accept loop 已呼叫 ActivateNrdc()。

② 轉送原始 bytes

ngapRaw 不必重新編碼。

③④ 拿回一個 IE

Secondary 的 DL $TEID_2$。

/> STATIC ③：SECONDARY 取出 UL TEID$_2$

一句話 同一則訊息，Secondary 只看「Additional」那一格。

gnb/xn.go · xnPduSessionResourceSetupProcessor（精簡）

```
1  ie.UnmarshalBinary(*item.…RequestTransfer, &transfer)
2  dlTeid, _ := g.teidGenerator.AllocateTeid()   // DL TEID₂
2  xnUe := NewXnUe(imsi, dlTeid, nil)
2  g.xnUeConns.Store(xnUe, struct{}{})
   for _, it := range transfer.ProtocolIEs.List {
3      add := it.AdditionalULNGUUPTNLInformation
       if add == nil { continue }
3      tnl := add.List[0].NGUUPTNLInformation
       gtp := tnl.Choice.(*ie.GTPTunnel)
4      xnUe.SetUlTeid(gtp.GTPTEID.Value)          // UL TEID₂
   }
```

① 解開 Transfer

和 Master 解的是同一則。

② 自己的 DL TEID$_2$

數值可能和 Master 相同，但位址不同。

③ Additional UL

Master 讀沒有 Additional 的那格。

④ 存進 XnUe

XnUe 只有資料面，沒有 N1。

/> STATIC ③：SECONDARY 回報 DL TEID$_2$

一句話 把自己的 DL TEID$_2$ 包成一個 IE 回給 Master，再把資料面查表登記好。

gnb/xn.go · xnPduSessionResourceSetupProcessor（精簡）

```
1 item := buildDCQosFlowPerTNLInformationItem(
1     xnUe.GetDlTeid(), g.ranN3Ip)
  // → GTPTunnel{TEID: DL TEID₂, Addr: 10.0.1.3}, QFI 1
  marshal, _ := ie.MarshalBinary(&item)
2 conn.Write(NewXnPdu(imsi, marshal).Marshal())   // → Master
3 g.dlTeidToUe.Store(hex(xnUe.GetDlTeid()), xnUe)
3 g.imsiTodlTeidAndUeType.Store(imsi, dlTeidAndUeType{
      dlTeid: xnUe.GetDlTeid(), ueType: UE_TYPE_XN})
3 g.markImsiReady(imsi)
```

① 一個 IE，不是整則 NGAP
Master 直接塞進 Response。

② 同一條 conn 回覆
Master 的 Read 隨即返回。

③ 和 setupN1 結尾一樣
只是 ueType 換成 xn。

/> STATIC ⑤：MASTER 組 RESPONSE

一句話 自己的 DL $TEID_1$ 放主欄位，Secondary 的 DL $TEID_2$ 放 Additional。

gnb/ngapBuilder.go（精簡）

```
   transfer := ie.PDUSessionResourceSetupResponseTransfer{
①      DLQosFlowPerTNLInformation: … dlTeid (Master) ,
   }
②  if nrdcIndicator {                        // = g.staticNrdc
       gtp, ok := qosItem.…Choice.(*ie.GTPTunnel)
③      if ok && gtp.GTPTEID != nil && gtp.GTPTEID.Value != nil {
④          transfer.AdditionalDLQosFlowPerTNLInformation =
④              &ie.QosFlowPerTNLInformationList{
④                  List: {qosItem}}
       }
   }
```

① 主欄位 = Master

DL $TEID_1$ @ 10.0.1.2。

②③ 兩道防線

設定要開，且 Secondary 真的回了 TEID。

④ Additional = Secondary

DL $TEID_2$ @ 10.0.1.3。

/> STATIC：IE 對照總整理

一句話 沒有 Additional 的歸 Master，有 Additional 的歸 Secondary。

IE	方向	誰配發	誰使用
UL NG-U UP TNL Info	SMF → Master	UPF（UL $TEID_1$）	Master ranUe
Additional UL NG-U UP TNL Info	SMF → Master → Xn	UPF（UL $TEID_2$）	Secondary xnUe
DL QoS Flow per TNL Info	Master → SMF	Master（DL $TEID_1$）	UPF FAR ①
Additional DL QoS Flow per TNL	Xn → Master → SMF	Secondary（DL $TEID_2$）	UPF FAR ②

記憶法：沒有 Additional 的歸 Master，有 Additional 的歸 Secondary。

/> STATIC ⑥⑦：UE 開兩條資料面

一句話 UE 同時連兩台 gNB 的資料面，各送一次 initial packet 報到。

ue/ue.go · connectToRanDataPlane（精簡）

```
1  conn, _ := util.UdpDialWithOptionalLocalAddress(
       u.ranDataPlaneIp, u.ranDataPlanePort, u.localDataPlaneIp)
1  conn.Write([]byte("initial packet imsi-" + u.supi))
2  if u.isNrdcEnabled() {
3      dc, _ := util.UdpDialWithOptionalLocalAddress(
           u.nrdc.dcRanDataPlane.ip, …port, …dcLocalDataPlaneIp)
       u.dcRanDataPlaneConn = dc
4      dc.Write([]byte("initial packet imsi-" + u.supi))
   }
   // setupTunnelDevice()：多開一條 DC reader → readFromRan
```

① Master 資料面

10.0.2.2 ⟶ 10.0.2.1:31414。

②③ Secondary 資料面

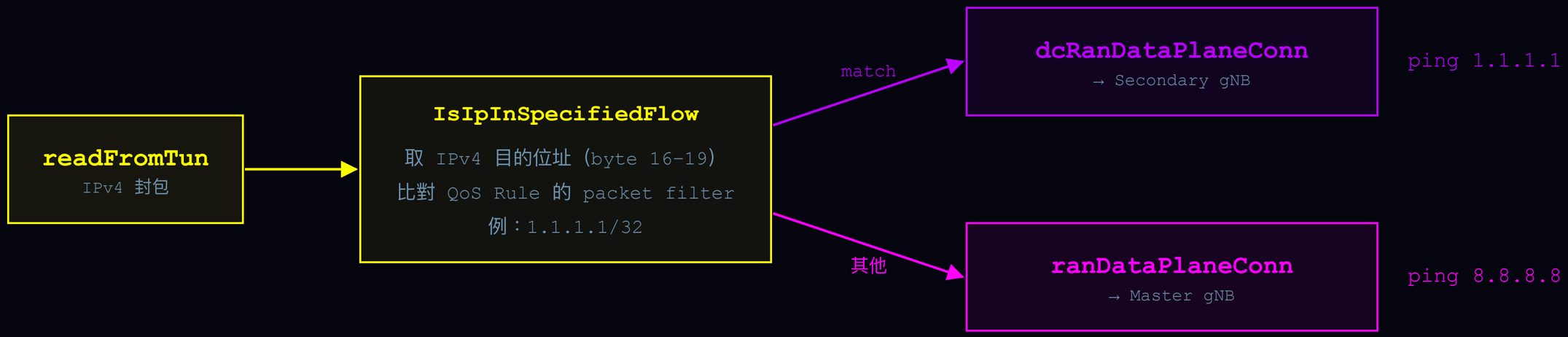
10.0.3.2 ⟶ 10.0.3.1:31414。

④ 同一個 IMSI 報到兩次

Master 綁到 ranUe、Secondary 綁到 xnUe。

UE 上行分流：SPECIFIEDFLOW

一句話 UE 看目的 IP 決定走哪台：符合規則走 Secondary，其他走 Master。



specifiedFlow 從哪來？

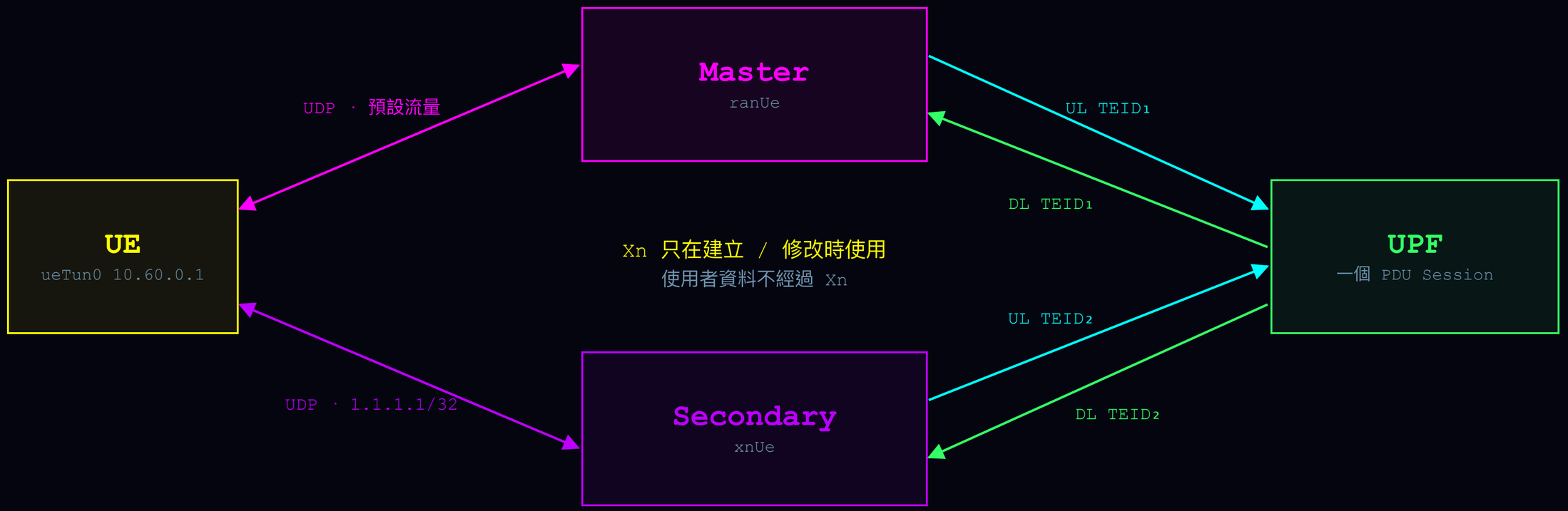
PDU Session Accept 的 Authorized QoS Rules → util.GetQosRule() 解析出的 packet filter。

為什麼只分上行？

下行由 UPF 決定走哪條隧道；UE 只要兩條連線都收、都寫回 TUN。

/> 結果：一個 SESSION、四個 TEID

一句話 一個 Session、兩台 gNB、四個 TEID；Xn 不傳使用者資料。



/> DYNAMIC

Session 建好之後，再由 Console 觸發：PDU Session Resource Modify Indication / Confirm

/> 觸發點：CONSOLE → GNB API

一句話 Console 打 Master 的 API，指名要切換哪個 UE。

gnb/gnb.go · handleConsoleGnbUeNrdcModify（精簡）

```
1  c.ShouldBindJSON(&request)        // {"imsi": "imsi-…"}
   var ranUe *RanUe
2  g.ranUeConns.Range(func(_, v any) bool {
       if v.(*RanUe).GetMobileIdentityIMSI() == request.Imsi {
           ranUe = v.(*RanUe)
       }
       return true
   })
3  if ranUe == nil { c.JSON(404, …); return }
4  err := g.processUePduSessionModifyIndication(ranUe)
   if err != nil { c.JSON(500, …); return }
   c.JSON(http.StatusOK, …)
```

① 同一支 API 開也關

方向由目前的 nrdcIndicator 決定。

②③ 只在 Master 上有效

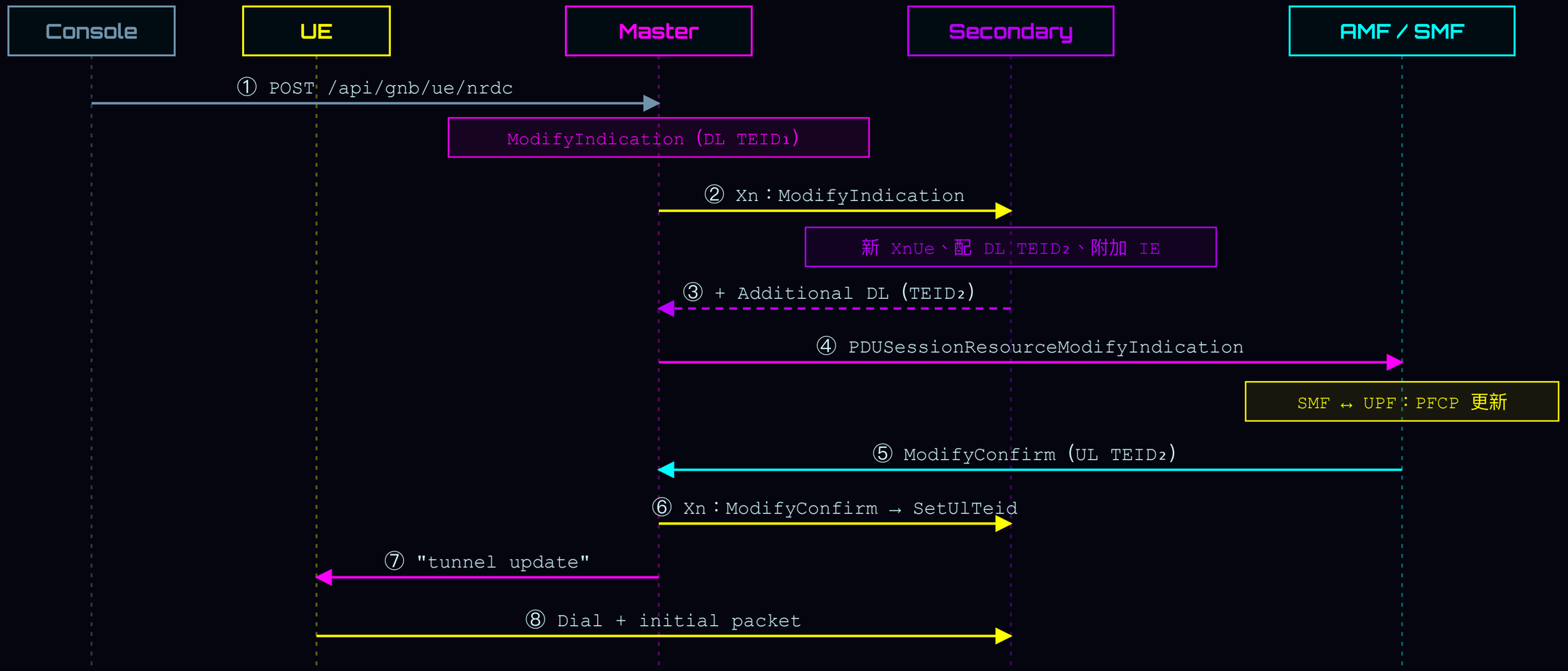
Secondary 找不到這個 UE → 404。

④ 同步等完成

最多 10 秒，成功才回 200。

/> DYNAMIC：開啟 DC

一句話 Console 觸發 → 兩次 Xn → 核心網同意 → 通知 UE 連上 Secondary。



/> DYNAMIC ①–④：MASTER 發起 MODIFY

一句話 Master 寫一則「我的隧道變了」，先給 Secondary 加料，再送 AMF。

gnb/gnb.go · processUePduSessionModifyIndication（精簡）

```
1  t, _ := getPDUSessionResourceModifyIndicationTransfer(
1      ranUe.GetDlTeid(), g.ranN3Ip, 1)        // DL TEID₁
2  ind, _ := getPDUSessionResourceModifyIndication(
       amfUeId, ranUeId, PDU_SESSION_ID, t)
3  ind, err = g.xnPduSessionResourceModifyIndication(imsi, ind)
   if err != nil { return err }
4  g.n2Conn.Write(ind)                          // → AMF
5  util.WaitComplete(
       ranUe.GetPduSessionModifyIndicationCompleteChan(),
       10*time.Second)
```

①② RAN 主動發起

ModifyIndication = gNB ⟶ AMF 的請求。

③ Xn 回來的就是要送的

開：附加了 $TEID_2$；關：原封不動。

⑤ 又一條 completion chan

等 dispatcher 收到 Confirm。

/> DYNAMIC ②③：SECONDARY 附加 $TEID_2$

一句話 已有 XnUe 就拆掉（關）；沒有就新建並把 $TEID_2$ 加進訊息（開）。

gnb/xn.go · …ModifyIndicationProcessor（精簡）

```
1  if xnReleaseUeProcessor(g, conn, imsi) {    // 關
1      conn.Write(XnPdu(imsi, ngapRaw)); return
   }
   ie.UnmarshalBinary(*item.…IndicationTransfer, &transfer)
2  dlTeid, _ := g.teidGenerator.AllocateTeid()  // DL TEID₂
2  g.xnUeConns.Store(NewXnUe(imsi, dlTeid, nil), struct{}{})
3  dcItem := buildDCQosFlowPerTNLInformationItem(
3      dlTeid, g.ranN3Ip)
3  transfer.AdditionalDLQosFlowPerTNLInformation = {dcItem}
4  ngapPdu := 重新 Marshal transfer → 塞回 → 整則重編
5  conn.Write(XnPdu(imsi, ngapPdu))
```

① 先嘗試釋放

回傳值就是方向。

②③ 開：附加 $TEID_2$

和 Static 用同一個 builder。

④ 直接改別人的訊息

解開 → 加 IE → 重新編碼。

UL $TEID_2$ 還不知道

要等 SMF 的 Confirm。

/> DYNAMIC ⑤–⑦：MASTER 收到 CONFIRM

一句話 核心網同意了：開的話把 Confirm 轉給 Secondary，然後通知 UE、切換狀態。

gnb/ngapDispatcher.go（精簡）

```
1 if msg.PDUSessionResourceFailedToModifyListModCfm != nil {
1     return                                  // 核心網拒絕
  }
2 if !ranUe.IsNrdcActivated() {               // 正在「開」
2     g.xnPduSessionResourceModifyConfirm(imsi, ngapRaw)
  }
3 ranUe.GetN1Conn().Write([]byte(UE_TUNNEL_UPDATE))
4 if ranUe.IsNrdcActivated() {
      ranUe.DeactivateNrdc()
  } else {
      ranUe.ActivateNrdc()
  }
```

小陷阱

函式名是 …ModifyIndicationProcessor，處理的卻是 Confirm。

② 只有「開」要轉 Confirm

Secondary 要從裡面拿 UL $TEID_2$。

③ "tunnel update"

自訂字串（非 3GPP），走 UE 控制面 TCP。

④ 切換狀態

最後 SignalComplete 叫醒 API handler。

/> DYNAMIC ⑥：SECONDARY 補上 UL TEID$_2$

一句話 從 Confirm 拿到 UL TEID$_2$，登記查表 —— 到這一步資料面才就緒。

gnb/xn.go · …ModifyConfirmProcessor（精簡）

```
    ie.UnmarshalBinary(*transferRaw, &confirmTransfer)
1   g.xnUeConns.Range(func(k, _ interface{}) bool {
1       if k.(*XnUe).GetIMSI() == imsi {
            xnUe = k.(*XnUe); return false
        }
        return true
    })
2   gtp, ok := confirmTransfer.ULNGUUPTNLInformation…
2   if ok { xnUe.SetUlTeid(gtp.GTPTEID.Value) }   // UL TEID2
3   conn.Write(NewXnPdu(imsi, []byte{}).Marshal()) // ACK
4   g.dlTeidToUe.Store(hex(xnUe.GetDlTeid()), xnUe)
4   g.markImsiReady(imsi)    // 以及 imsi → (teid, xn)
```

① `return false` = 找到就停

sync.Map 的慣用法。

② UL TEID$_2$

SMF 在 Confirm 裡告訴我們。

③ 空的 XnPdu = ACK

只是說「收到了」。

④ 比 Static 晚登記

等 Confirm 才開放資料面。

/> DYNAMIC ⑦⑧：UE UPDATEDATAPLANE

一句話 UE 收到 "tunnel update"：沒開就連上 Secondary，開著就把它關掉。

ue/ue.go（精簡）

```
1  case UE_TUNNEL_UPDATE: go u.updateDataPlane()

   func (u *Ue) updateDataPlane() {
2      u.rwLock.Lock(); defer u.rwLock.Unlock()
       if !u.nrdc.enable {                          // 開
3          conn, _ := util.UdpDialWithOptionalLocalAddress(…)
3          conn.Write([]byte("initial packet imsi-" + u.supi))
           go func() { /* DC reader → readFromRan */ }()
           u.dcRanDataPlaneConn, u.nrdc.enable = conn, true
       } else {                                     // 關
4          u.dcRanDataPlaneConn.Close()
           u.nrdc.enable = false
       }
   }
```

① waitForRanMessage 收到

控制面 TCP 上的字串訊息。

② 寫鎖

handleDataPlane 用讀鎖讀 enable。

③ 開：Dial + 報到

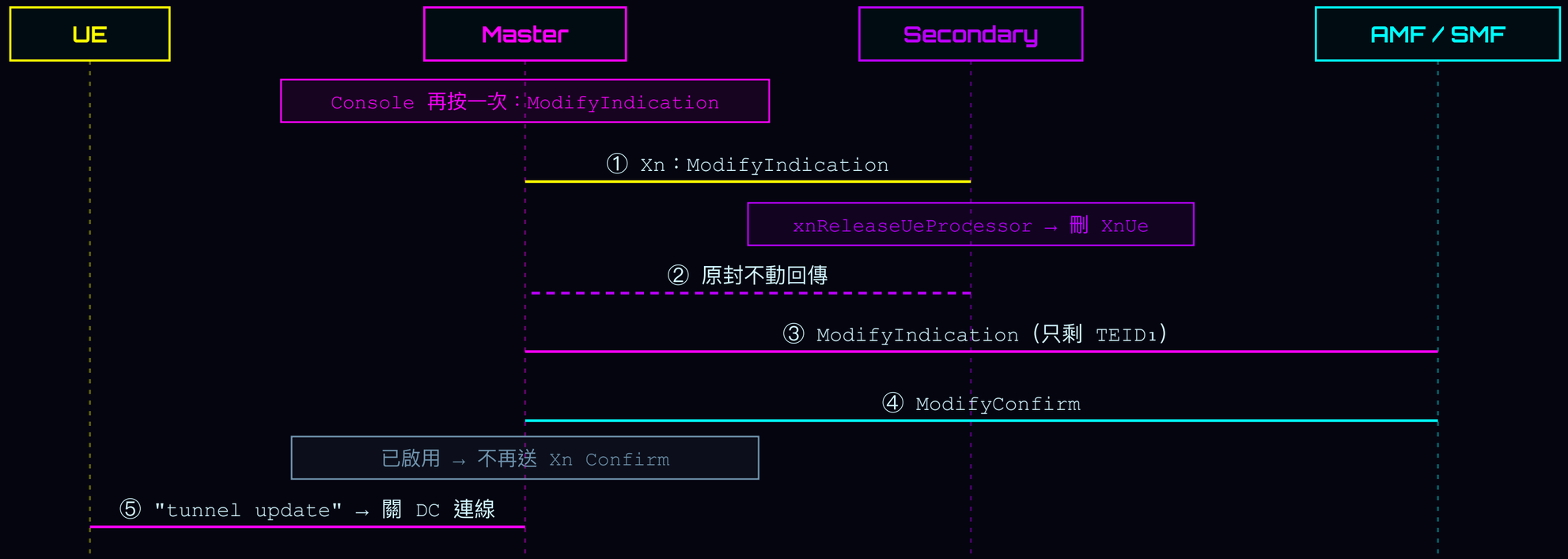
和 Static 的第二條連線一模一樣。

④ 關：Close

DC reader 收到 ErrClosed 自己結束。

/> DYNAMIC：關閉 DC

一句話 關閉只要一次 Xn：先拆 Secondary，再請核心網移除第二條隧道。



關閉只需要一次 Xn：Secondary 先把 XnUe 拆掉，Master 再請核心網把第二條隧道移除。

/> XnReleaseUeProcessor：拆掉 XnUe

一句話 建立時登記了什麼，關閉時就一樣一樣清掉。

gnb/xn.go · xnReleaseUeProcessor（精簡）

```
func xnReleaseUeProcessor(g, conn, imsi) bool {
    xnUe := 依 imsi 在 xnUeConns 找
①   if xnUe == nil {
①       return false     // 沒有 → 這次是「開」
    }
②   g.dlTeidToUe.Delete(hex(xnUe.GetDlTeid()))
②   g.addressToUe.Delete(xnUe.GetDataPlaneAddress()…)
②   xnUe.Release(g.teidGenerator) // 歸還 TEID₂
②   g.xnUeConns.Delete(xnUe)
③   return true          // 有 → 已拆掉，是「關」
}
```

①③ 回傳值就是方向

false = 開、true = 關。

② 四樣都要清

和建立時一一對應。

Master 端不送 Confirm

收到 Confirm 時仍是啟用狀態，直接通知 UE 並 DeactivateNrdc。

/> NR-DC 狀態機

一句話 只有兩個狀態，方向全靠一個 bool 判斷。



判斷方向只靠一個 bool：dispatcher 收到 Confirm 時看 IsNrdcActivated()

UE 端同理：updateDataPlane() 依 nrdc.enable 決定開或關

/> 動手做：STATIC NR-DC

一句話 起兩台 gNB、一個 UE，ping 兩個不同目的地，看封包走不同路。

terminal

```
git clone https://github.com/free-ran-ue/free-ran-ue.git
cd free-ran-ue && make
# 機器 / namespace A
1 ./build/free-ran-ue gnb -c config/gnb-dc-static-master.yaml
# 機器 / namespace B
1 ./build/free-ran-ue gnb -c config/gnb-dc-static-secondary.yaml
2 sudo ./build/free-ran-ue ue -c config/ue-dc-static.yaml
ifconfig ueTun0                    # inet 10.60.0.1
3 ping -I ueTun0 8.8.8.8 -c 5      # 走 Master
3 ping -I ueTun0 1.1.1.1 -c 5      # 走 Secondary
```

① 兩台 gNB 不同機器

都要綁 UDP 2152，同機會衝突。

② UE 不和 free5GC 同機

可用 script/namespace-script 切 namespace。

③ 先加 Flow Rule

Webconsole 的訂閱者加上 1.1.1.1/32。

/> 動手做：DYNAMIC NR-DC

一句話 先單連線跑起來，再從 Console 或 curl 切換，觀察 1.1.1.1 改走哪台。

terminal

```
./build/free-ran-ue gnb -c config/gnb-dc-dynamic-master.yaml
./build/free-ran-ue gnb -c config/gnb-dc-dynamic-secondary.yaml
sudo ./build/free-ran-ue ue -c config/ue-dc-dynamic.yaml
./build/free-ran-ue console -c config/console.yaml
# 瀏覽器：admin / free-ran-ue → Add gNB (:40104)
#   Master 資訊頁 → 打開 UE 的 NR-DC 開關
# 或直接打 Master 的 API：
curl -X POST http://10.0.1.2:40104/api/gnb/ue/nrdc \
  -H 'Content-Type: application/json' \
  -d '{"imsi": "imsi-208930000000001"}'
```

1. **開 DC 前**
   ping 1.1.1.1 也走 Master
2. **切換**
   UE log：updated to NRDC mode
3. **開 DC 後**
   1.1.1.1 改走 Secondary
4. **再切一次**
   XnUe 消失、回到單連線

/> 除錯小抄

一句話 看 log tag、抓封包、打 API —— 三招定位問題。

LOG

`logger.level: "trace"`

印出每個 NGAP / NAS / GTP 封包的 bytes；平常用 info 或 debug

TAG

XN · GTP · NGAP · NAS

依 log tag 過濾：DC 問題先看 XN，資料面先看 GTP

PCAP

`tcpdump -i any sctp or port 2152`

Wireshark 看 NGAP 的 Additional IE、GTP 的 TEID

API

`GET /api/gnb/info`

確認 RanUe 的 nrdcIndicator、Secondary 有沒有 XnUe

常見錯誤：No UE found for DL TEID（查表沒登記）、timed out waiting for dl teid（initial packet 等不到 markImsiReady）。

/> SUMMARY

01 **RAN = CU + DU + RU**

F1、E1、前傳 7.2x；Small Cell 整合、O-RAN 開放、OCUDU 開源

02 **free-ran-ue：Uu 換成 socket**

TCP 傳 NAS、UDP 傳 IP；N2 / N3 完全標準

03 **gNB：上行一 UE 一 goroutine，下行一個 dispatcher**

用 RAN-UE-NGAP-ID 配對、用 completion chan 同步

04 **資料面：userspace GTP-U + 四張查表**

initial packet 綁位址、TEID 找 UE

05 **UE：TUN + channel + select**

註冊、建 session 後才開資料面

06 **NR-DC：Xn 只換 TEID**

Static 走 Setup、Dynamic 走 Modify；Additional IE 歸 Secondary